Your First JavaScript Line

You will learn how to write to the console in JavaScript.

Learning programming often begins with executing a **“Hello World!”** program. In some languages like Java, it takes a lot to write Hello World to the standard output. In JavaScript, we get away with one single line:

```
console.log( "Hello World");
```

Easy Peasy.

Let’s try writing it differently

Notice your statement can span multiple lines. I have pressed enter inside the expression to separate content into two lines. You can run your reformatted code in the code editor below:

```
console
   .log(
        "Hello world!"
);
```

As you can see, you can format JavaScript code in any way you want. The interpreter will not care about the redundant whitespace characters.

Instead of “Hello World!”, I have written 5 + 2. Run and check what do you see. Experiment a bit more with the log in the code editor below:

```
console.log( 5 + 2 );
```

Printing more than one value

`console.log` may print any number of arguments separated by commas. In the console, the values appear next to each other separated by a space.

```
console.log("Hi", "Hey", "Hello :)");
```

Congratulations! You managed to write Hello World! to the console. Let's see what we learned:

> 1. console.log writes a log message to the console.
> 2. "Hello World!" is a string. One way to formulate a string is using double quotes. Mind you, 'Hello World!' is also a valid string notation in JavaScript.
> 3. there is a semicolon at the end of the statement. The semicolon itself is optional, but I recommend using it.
> 4. You can write more than one value to the console which will be displayed separated by space.

Comments

You will learn how to write comments that will help you to remember what code you wrote and why you wrote it that way.

We often place comments in our code so that we know what it is doing.

JavaScript is similar to C++ and Java in its commenting style:

- Everything between `/*` and `*/` is ignored by the interpreter.
- Everything after `//` on the same line is ignored by the interpreter.

```
let a = 5 + 2; // This is a single line comment lasting until the end of the line.

/*
     A comment that can span multiple lines.
     Everything inside the comment will be ignored by the JavaScript interpreter.
     let b = 2;
*/
```

The above two comments are interpreted as:

```
let a = 5 + 2;
```

Everything else is ignored.

Why do we use comments? To avoid ending up in similar situations:

```
// Dear maintainer:
//
// Once you are done trying to 'optimize' this routine,
// and have realized what a terrible mistake that was,
// please increment the following counter as a warning
// to the next guy:
//
// total_hours_wasted_here = 42
//
```

My other favorite is:

My other favorite is:

```
// I dedicate all this code, all my work, to my wife, Darlene, who will
// have to support me and our three children and the dog once it gets
// released into the public.
```

Try to write readable code with good comments.

Data Types and Arithmetic Operators

This lesson will introduce you to data types to represent data and also to basic arithmetic operations you can perform with them.

Introduction

Most programming languages help you create values that symbolize a number, a character in a text, or a longer text. You can also symbolize the concept of true and false values using booleans. You can also create values that symbolize the absence of a value. These values are all called *primitive datatypes*.

The name primitive does not come from a negative place. These datatypes are neither stupid, nor inferior to any other datatypes we use in JavaScript. The name primitive comes from their simplicity. The other datatypes you will learn later are *composite*, and they consist of many primitive datatypes.

Primitive Types

In JavaScript, there are six primitive types:

1. boolean (`true` or `false`)
2. number (including integers like `1`, `-2`, and floating point numbers like `1.1`, `2e-3`)
3. string (`‘’` or `“”`, `‘ES6 in Practice’` or `“ES6 in Practice”`)
4. null type (denoted by `null`)
5. undefined (denoted by `undefined`)
6. Symbol (don’t worry about them yet)

Integers and Arithmetic operators

Let’s try some *Javascript expression*.

```
console.log(5+2);
console.log(7%5);
```

```
console.log(5**2);
```

Handling integers is straightforward. You have the four arithmetic operations (`+`, `-`, `*`, `/`) available for addition, subtraction, multiplication, and division respectively.

The `%` operator is called modulus. `a % b` returns the remainder of the division `a / b`. In our example, `7 / 5` is `1`, and the remainder is `2`. The value `2` is returned.

The `**` operator is called the exponential operator. `5 ** 2` is five raised to the second power.

Floating Numbers

Let's run the code below.

```
console.log(0.1+0.2);
console.log(3.1e-3);
```

Due to the way how numbers are represented, `0.1 + 0.2` is not exactly `0.3`. This is normal and occurs in most programming languages.

`3.1e-3` is the normal form of `0.0031`. Read it like the exact value of 3.1 times ten to the power of minus three. Although the form is similar to `3.1 * (10 ** -3)`, there are subtle differences. `3.1e-3` describes the exact value of `0.0031`. `3.1 * (10 ** -3)` describes a composite expression that needs to be calculated:

```
console.log(3.1 * (10 ** -3))
```

Even floating point arithmetics does not even make it exact.

NaN

The division `0 / 0` or using mismatching types creates a special number called *not a number* or `NaN`.

```
console.log(0 / 0);
console.log('Javascript in Practice'*2);
```

The latter in the code example above is interesting to Python users because, in Python, the result would have been `'Javascript in PracticeJavascript in Practice'`. JavaScript does not work like that.

Infinity Type

There is another interesting type: infinity.

```
console.log(1 / 0);
console.log(Infinity * Infinity);
console.log(-1 / 0);
console.log(1e+308);
console.log(1e+309);
```

JavaScript registers very large numbers as infinity. For instance, ten to the power of 309 is represented as infinity. Division by zero also yields infinity.

Now, this was just an overview of some data types. We will explore more of them in the future.

Playing Around With Strings

Let's dive into Strings and see how can you concatenate them. This lesson will also introduce you to Automatic Type Casting and you'll learn how to parse Strings as Integers and Floats.

A string is basically a series of characters.

Concatenating strings

Let’s perform some operations on strings.

```
console.log('Javascript in ' + 'Practice')
```

The plus operator concatenates strings. Concatenation means that you join the contents of two strings one after the other.

Strings are *immutable* which means that their value cannot be changed. When concatenating two strings, the result is saved in a third string.

Automatic Type Casting

If any of the operands of plus is an integer, the result becomes a string. JavaScript automatically converts the operands of an operator to the same type. This is called *automatic type casting*:

```
console.log("1 + \'2\' becomes",1 + '2');
console.log("\'1\' + 2 becomes",'1' + 2);
```

Rules may become confusing, so don’t abuse automatic type casting.

Parsing Strings as Integers

Just know that you may have to explicitly cast a string to an integer to be able to add it to another integer:

```
console.log("1 + + \"2\"")
console.log(1 + +"2") // +"2" gives a sign to "2", converting it to a number

console.log("1 + Number(\"2\")")
console.log(1 + Number("2"))

console.log("1 + Number.parseInt( \"2\", 10 )")
console.log(1 + Number.parseInt( "2", 10 ))

console.log("1 + Number.parseInt( \"2\" )")
console.log(1 + Number.parseInt( "2" ))
```

All the above conversions work. The first relies on giving a sign to a numeric string which converts it to a number. Then `1+2` becomes `3`. The second type cast is more explicit: you use `Number` to wrap a string and convert it to a number.

I recommend using the third option: `Number.parseInt` with a radix. `parseInt` converts a string into a number. The second argument of `parseInt` is optional: it describes the base in which we represent the number.

```
console.log("Number.parseInt(\"JS in Practice\")");
console.log(Number.parseInt("JS in Practice"));

console.log("Number.parseInt( \"10\", 2 )");
console.log(Number.parseInt( "10", 2 ));

console.log("Number.parseInt( \"a\" )");
console.log(Number.parseInt( "a" ));

console.log("Number.parseInt( \"a\", 16 )");
console.log(Number.parseInt( "a", 16 ));
```

Arbitrary strings are often `NaN`. “2” in base 2 is 10. You can see how easy it is to convert a binary or a hexadecimal (base 16) string into a decimal number. Base 16 digits are `0123456789abcdef`. The last 6 digits may also be upper case.

`Number.parseInt` recognizes the starting characters of a string as integer numbers, and throws away the rest:

```
console.log(Number.parseInt( "1234.567 89" ));
```

Parsing Strings as Floating Numbers

The dot is not a character present in integer numbers, so everything after `1234` is thrown away by `Number.parseInt`.

You can also use `Number.parseFloat` to parse floating point. It parses the floating point number until the terminating space:

```
console.log(Number.parseFloat( "1234.567 89" ));
```

Comparison and Ternary Operators

You'll look at how comparison and ternary operators are used to evaluate conditions and boolean values.

Let's see some booleans values. Booleans are either true or false.

Comparison Operators

We can compare two numbers with `>`, `>=`, `==`, `===`, `<=`, `<`. We will discuss the difference between `==` and `===` soon. For the rest of the operators, the result of the comparison is a boolean.

The ! operator

The `!` operator negates its operand. True becomes false, and false becomes true.

> *A **truthy** value is a value `v` for which `!!v` is `true`. Examples of truthy values are: nonzero integers, strings containing at least one character.*
> *A **falsy** value is a value `w` for which `!!w` is `false`. Examples of falsy values are: empty string, `0`, `null`, `undefined`.*

We can convert a value to a boolean by negating it twice: `!!`:

- assuming `v` is truthy, `!v` becomes `false`. `!!v` becomes `!false`, which becomes `true`.
- assuming `w` is falsy, `!w` becomes `true`, and `!!w` becomes `false`.

```
console.log("5 <= 5", 5 <= 5);
console.log("5 < 5",5 < 5);
console.log("!(5 < 5)",!(5 < 5));   // ! stands for negation. !true = false, !false = true.
console.log("!!\"\"",!!"")
console.log("!!\"a\"",!!"a")
console.log("!!0",!!0)
console.log("!!1",!!1)
console.log("!!NaN",!!NaN)
console.log("!!Infinity",!!Infinity)
console.log("!!null",!!null)
```

```
console.log("!!undefined",!!undefined)
```

Equal to

For values `a` and `b`, `a == b` is true if and only if both `a` and `b` can be converted to the same value via type casting rules. This includes:

- `null == undefined` is true
- If an operand is a string and the other operand is a number, the string is converted to a number
- If an operand is a number and the other operand is a boolean, the boolean is converted to a number as follows: `true` becomes `1`, and `false` becomes `0`.

Don't worry about the exact definition, you will get used to it.

Equal Value and Equal Type

For values `a` and `b`, `a === b` is `true` if and only if `a == b` and both `a` and `b` have the same types.

```
console.log("5 == \'5\'",5 == '5');   // '5' is converted to 5
console.log("5 === \'5\'",5 === '5');  // types have to be the same
console.log("0 == \'\'",0 == '');    // '' is converted to 0
console.log("0 === \'\'",0 === '');   // types have to be the same
console.log("NaN == NaN",NaN == NaN); // I know... just accept this as something odd and funr
```

Not equal to

The negation of `==` is `!=`. Read it as *is not equal to.*

The negation of `===` is `!==`.

```
console.log("5 != \'5\'",5 != '5')
console.log("5 !== \'5\'",5 !== '5')
```

Ternary Operator

Let me introduce the ternary operator to drive home a strong point about truthiness.

The value of `a ? b : c` is:

- `b` if `a` is truthy
- `c` if `a` is falsy

It is important to note the difference between `2 == true` and `!!2`.

```
console.log("2 == true",2 == true)    // true is converted to 1
console.log("!!2",!!2)          // 2 is a truthy value
console.log("2 == true ? \'the condition is true\' : \'the condition is false\'")
console.log(2 == true ? 'the condition is true' : 'the condition is false')
console.log("!!2 ? \'the condition is true\' : \'the condition is false\'")
console.log(!!2 ? 'the condition is true' : 'the condition is false')
```

I have seen the nastiest bug in my life in a code, where a condition was in a format `num == true`. As I never felt like learning boring definitions, my lack of knowledge shot me in the foot, because I assumed the opposite conversion in `2 == true`. I can save you some headache by highlighting this common misconception. In `2 == true`, `true` is converted to `1`, and not the other way around.

Null, Undefined and Symbol

Let's introduce you to the Null, Undefined and Symbol datatypes.

Null, undefined, and Symbols are primitive types.

Null

Null represents an intentional absence of a primitive or composite value of a defined variable.

Undefined

Undefined represents that a value is not defined.

Symbol

A `Symbol()` is a unique value without an associated literal value. They are useful as unique keys, because `Symbol() == Symbol()` is false. At this stage, just accept that symbols exist. You don't have to use them for anything yet.

```
console.log(null);
console.log(undefined);
console.log(void 0);
console.log(Symbol('ES6 in Practice'));
```

`void` is a keyword in Javascript which specifies that the expression next to it will not return anything. You don't need to worry much about it right now.

The let keyword

The let keyword will allow you to store data for future reference.

Declaration

We can create variables with the `let` keyword. Think of a variable like a drawer. `let` *declares* a variable, which means to you that a drawer is created with a handle.

```
let myDrawer;
```

You can put a value in your drawer:

```
myDrawer="$1.000";
```

In order to access the value, you have to grab the handle of the box and open it. In this example, you have a drawer called `myDrawer`. It contains a string written `'$1.000'` on it.

To access your thousand bucks, you have to open the drawer:

`myDrawer`
`'$1.000'`

Initialization

You can assign an initial value to your variable with the `=` sign. This is called *initialization*, and it can occur either in the same statement where you declared the variable (see `x`), or after the declaration (see `y`). You may access a declared variable even if you have not initialized it. Its value becomes `undefined`.

```
let x = 5;
let y;

y = x ** 2;
let z;
console.log( x, y, z );
```

Move `let z` below the `console.log` statement. You should see a ReferenceError:

`ReferenceError: z is not defined`

The message is somewhat misleading, because it means `z is not declared` using the `let` keyword. Don't mix this message with the `undefined` value. You only get the above reference error if you reference a variable that does not exist.

You did the following: you asked for the contents of your drawer `z` in the console log. But the drawer itself does not exist yet. You created the drawer afterwards, with `let z;`.

Side note: I know, in most tutorials, you see var instead of let. This is an advantage of reading an ES2018-compliant tutorial. Don't worry about var for now, you will hear about it later. Ok, I understand. If you do worry about it, read this article.

The Object Type

A brief overview for the Object data type

An object is data structure with String keys and arbitrary values. Imagine it like a machine that accepts a key, and gives you a value. Some people call this data structure *associative array*, others call it *hashmap*. These names may sound fancy, but essentially we mean the same damn thing.

Side note: `Symbol` keys are also allowed for strings. This is an advanced topic, you will learn it later.

```
let author = {
  name: null,
  website: "zsoltnagy.eu",
  age: 35
}

author.name = "Zsolt";
console.log( author.name, author[ "website" ] )
// Prints: Zsolt zsoltnagy.eu

delete author.name
console.log( "Author name: ", author.name )
// Prints: Author name: undefined
```

A *field* in an object can be referenced in the following ways:

- dot notation: `object_name.member_name`
- bracket (associative array) notation: `object_name[ string_or_symbol_value ]`

The `delete` operator deletes a field from an object.

If a field in an object is deleted or not even declared, the value of this field is `undefined`.

We will learn a lot more about objects later.

Introduction to Arrays

Arrays provide us with a way to create and maintain a list of data. In this lesson, you'll be introduced to arrays and how you can access the elements inside them.

Introduction

An array is an ordered list of items. The items may be of any type. You know, in most post offices, there are hundreds or thousands of post boxes. Each post box may or may not contain something. Each post box has a numeric handle. Post box 25 may be your box. You unlock it, grab its handle, and access its contents.

The trick is that in case of arrays, you have to imagine the post boxes with keys 0, 1, 2, and so on. Typically, arrays have continuous keys.

```
let days = [ 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday' ];
```

Arrays do not have to contain elements of the same type:

```
let storage = [ 1, 'Monday', null ];
```

Accessing elements

Each element of the array can be accessed using an index starting from zero. The syntax for indexing is shown below:

```
let days=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'];

console.log(days[0]);
console.log(days[4]);
console.log(days[5]);
```

In the third example, we indexed out of the array. As we indexed beyond the

In the third example, we indexed out of the array. As we indexed beyond the length of the array, the program returned undefined for `days[5]`.

Array length

Arrays have lengths:

```
let days=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'];
console.log(days.length);
```

Adding and Removing Elements

After being introduced to the concept of arrays, we'll now learn how to add elements to them and how to remove elements from them.

Adding elements to the beginning or end

You can add elements to the beginning and to the end of the array:

- Push adds elements to the end of the array
- Unshift adds elements to the beginning of the array

```
let days=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'];

days.push( 'Saturday' );
console.log("Saturday is added to the end by the push() function\n", days );
days.unshift( 'Sunday' );
console.log("Sunday is added to the beginning by the unshift() function\n", days );
```

Removing elements

You can also remove these elements from the array:

- Pop removes the last element from the array and returns it.
- Shift removes the first element from the array and returns it.

```
let days=['Sunday','Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'];
//pop
let element = days.pop();
console.log( element, days );
 //shift
let secondElement = days.shift();
console.log( secondElement, days );
```

Similarly to objects, you can `delete` any element from the array. The value `undefined` will be placed in place of this element:

```
let days=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
delete days[2]
console.log(days)
```

Overwriting and Adding Elements At Any Index

The values of an array can be set by using their indices, and equating them to a new value. You can overwrite existing values, or add new values to the array. The indices of the added values do not have to be continuous:

```
let days=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
days[2] = 'Wednesday';
days[9] = 'Wednesday';
console.log( days );
```

As with most topics, bear in mind that we are just covering the basics to get you started in writing code. There are multiple layers of knowledge on JavaScript arrays. We will uncover these lessons once they become important.

Exercise 1

Suppose the following array is given:

```
array = [ 7, 15, 32, 9, '3', '11', 2 ];
```

Determine the output printed to the Javascript console.

All questions are part of the same program. Each instruction in every question is in sequence with the previous instructions.

1

```
console.log( array[3] );
```

2

```
console.log( array[7] );
```

3

```
console.log( array.pop() );
```

4

```
console.log( array );
```

5

```
console.log( array.shift() );
```

6

```
console.log( array );
```

7

```
console.log( array.push( 6 ) );
```

8

```
console.log( array);
```

Check Answers

Run the code below to confirm the solutions :

```
let array = [ 7, 15, 32, 9, '3', '11', 2 ];

console.log( array[3] );
console.log( array[7]);
console.log( array.pop() );
console.log( array );
console.log( array.shift() );
console.log( array );
console.log( array.push( 6 ) );
```

```
console.log( array.push( 6 ) );
console.log( array );
```

Functions

A function is basically a piece of code which can be reused without having to write it again and again. Let's discover it and learn how to create, refer to and call a function.

The concept of function

Think of a function like a mathematical function giving you a relationship between input and output variables. If you don't like maths, think of a function like a vending machine. You give it some coins and a number, and it spits out some cold coke.

```
function add( a, b ) {
    return a + b;
}
```

This function definition describes the relationship between its input variables `a` and `b`, and the return value of the function.

The return statement returns the value of the function. When calling the `add` function with arguments `a` and `b`, it computes the value `a+b` and returns it. Example:

```
function add( a, b ) {
  return a + b;
}
console.log(add( 5,2));
```

Try to modify the input variables while making the function call `add(5,2)`. The return value also changes. Try to call the add function with one variable, e.g. `add( 5 )`, and see what happens.

Functions are useful to create reusable chunks of code that you can call with different arguments. We will write more useful functions once you learn the

basics of control structures.

Making functions using a variable as a reference

You can also define functions without placing the name between the `function` keyword and the argument list. This structure is great if you want to create a reference to it using a variable. Remember, the variable `subtract` is a handle to a drawer. This time, your drawer contains a function.

```
let subtract = function( a, b ) {
    return a - b;
}
```

Calling functions

All functions can be called using their references:

```
let add=function( a, b )
{
  return a+b;
}

let subtract=function( a, b ) {
  return a-b;
}

let multiply=function( a, b ) {
  return a*b;
}

let square=function( a ) {
  return a**2;
}
console.log(add( 2,3));
console.log(subtract( 2, 3 ));
console.log(multiply( 2, 3 ));
console.log(square( 2 ));
```

What if the function does not return anything?

When a function does not return anything, its return value becomes `undefined`:

```
let empty = function() {}
```

```
console.log(empty())
```

Function Scope

Variables inside the function are not the same as variables outside the function:

```
let a = 5;
function addOne( a ) {
    a = a + 1;
    return a;
}

console.log( addOne( a ))
console.log(a)
```

The `a` variable inside the function is valid inside the *scope* of the function. It *shadows* the variable `a` outside the function. Therefore, adding one to the internal `a` variable does not have any effect on the external value. Therefore, changes in the value of `a` inside the function do not change the value of the variable `a` outside the function.

The typeof operator

In this lesson, you'll learn about the typeof operator which will tell you if you are you dealing with a number or some text or even if there is a value at all.

Congratulations! You now know the basic types of JavaScript. This is a perfect time to introduce an operator that returns the type of a value.

The `typeof` operator accepts one operand and returns a string. This string describes the type of an object.

```
typeof ''          // "string"
typeof 0           // "number"
typeof true        // "boolean"
typeof {}          // "object"
typeof []          // "object"
typeof null        // "object"
typeof undefined   // "undefined"
typeof Symbol()    // "symbol"
```

Differentiating between arrays, objects and null

There is just one problem: arrays, objects, and the `null` value have the same type. It would be great if we could differentiate between them. Fortunately, ES6 solves the array problem:

```
console.log(Array.isArray( [1, 2] ))
console.log(Array.isArray( {a: 1} ))
```

Regarding the null value, a simple comparison does the trick:

```
let name = null;
console.log( name === null)
```

typeof Function

Functions have the type `“function”`:

```
function two() { return 2; }

console.log(typeof two)
```

Some more operators

Enrich your code with more operators that will help you change values and formulate better conditions.

We will soon write conditions and loops. In these two control structures, a few operators come handy:

- `+=`, `-=`, `*=`, `/=`, `%=` etc. are abbreviations for adding a value to a variable. `a += 1` is the same as writing `a = a + 1`.
- `++x` increases the value of `x` by `1`, then returns the increased value
- `x++` returns the original value of `x`, then increases its value by `1`
- `--x` decreases the value of `x` by `1`, then returns the decreased value
- `x--` returns the original value of `x`, then decreases its value by `1`

++x and x++

I know, the difference between `++x` and `x++` may not make sense to you right now. I argue that in most cases, it should not even make a difference as long as you want to write readable code.

Both `x++` and `++x` have a *main effect* and a *side effect*. As a main effect, they return a value. Either `x`, or `x + 1`, depending on whether you write the ++ after the x or before. Basically the difference comes because of precedence,

- `x++` executes the statement and then increments the value.
- `++x` increments the value and then executes the statement.

```
let a = 1;
a *= 2; // a = a * 2;
console.log( '2++: ', a++ );
console.log( '--3: ', --a );
```

`console.log(a++)` writes the value of `a` to the console before incrementing it

`console.log(a++)` writes the value of `a` to the console before incrementing it. The value of `a` was `2` after the second line of the program. After the value of

`a` is written to the console, the value of `a` is incremented and it now becomes 3. Then `console.log(--a)` first decrements the value making it 2 and then writes it to the console.

Quiz Time!

Q

Determine the value of `a` at the end of this program.

```
var a = 3;
++a;
a *= 3;
a %= 5;
```

COMPLETED 0%

1 of 1

The && operator

There are two more operators that come handy when formulating boolean conditions.

Suppose you are checking whether a variable is an array **and** it has a positive length. The **and** conjunction is denoted by the `&&` operator:

```
function isNonEmptyArray( x ) {
    return Array.isArray( x ) && x.length > 0;
}

console.log(isNonEmptyArray( 5 )) //false not an array and does not have positive value
console.log(isNonEmptyArray( [] )) //false
console.log(isNonEmptyArray( [1] ))//true
```

- `isNonEmptyArray(5)` returns `false` because `5` is not an array and does not have a positive value
- `isNonEmptyArray([])` returns `false` because it is an array but does not have any positive value
- `isNonEmptyArray([1])` returns `true` because it satisfies both the conditions as `[1]` is an array and it also contains a positive value

The and *shortcut*

Also, please note that the value `5` is not an array, so `Array.isArray( 5 )` becomes false. A `false` value in conjunction with anything is `false`:

```
> false && false
false
> false && true
false
```

Therefore, the second operand of the `&&` operator is not even executed. This is good, because `5.length` would have *thrown an error*. We can rely on the `&&` operator not executing the right hand side expression in case the left side is evaluated as `false`. This simplification is called a *shortcut*.

If the left hand side is true, the right hand side is executed. In this case, the value of the `and` expression becomes the value of the right hand side expression:

```
 > true && false
false
> true && true
true
```

The || operator

We can also define an **or** relationship between operands. For instance, suppose we have a function that returns true for numbers and strings:

```
function numberOrString( x ) {
    return typeof x === 'number' || typeof x === 'string'
}

console.log(numberOrString( NaN )) //true
console.log(numberOrString( '' )) //true
console.log(numberOrString( null )) //false
```

- `numberOrString( NaN )` returns `true` because it has the `number` type
- `numberOfString( ‘’ )` returns `true` because it has the `string` type
- `numberOrString( null )` returns `false` because it does not match the `number` nor the `string` type

The or *shortcut*

The or operator works as follows: in case of `a || b`, if `a` is `true`, then `b` is not even evaluated. This is the or *shortcut*. Once we know that a variable is a number, we don’t have to check if the same variable is a string too. We already know that the function should return `true`.

How to check for NaN value?

The value `NaN` was mentioned. When formulating a condition that checks if a value is `NaN`, we have the problem that `NaN == NaN` is false. The value `NaN` does not equal to itself. How can we then check if a value is `NaN`? The answer is, use `Number.isNaN()`.

```
console.log(Number.isNaN( NaN ))
```

Exercise 2

1. Without running the code, determine the values of the below variables. You can write your answers as comments in the code editor and then compare them afterwards with the program output.

```
let a = typeof '';
console.log(a);
let b = typeof a;
console.log(b);
let c = typeof x;
console.log(c);
let d = typeof 25;
console.log(d);
let e = typeof null;
console.log(e);
let f = typeof 25 + ' ';
console.log(f);
let g = typeof (NaN === NaN);
console.log(g);
let h = typeof null;
console.log(h);
let i = '' + 1;
console.log(i);
let j = typeof i;
console.log(j);
```

Exercise 3

Create a function that accepts an array of numbers as an argument. Return the maximum value in the array.

Example:

```
console.log(max([19, 65, 9, 17, -99]));
> 65
console.log(max([]));
> -Infinity
```

```
function max( arr ){
  //Write your code here
}
```

If, Else If and Else Statements

You will now learn how to branch off based on conditional statements. This lesson will teach you about if, else if and else statements.

The time has come, when we can fit the puzzle pieces together. You have learned about different primitive and composite types. You know how to check these types. You know the basic functions and arithmetic (addition, subtraction, multiplication etc.) and logical (true or false) operations in JavaScript.

There are only two things missing.

So far, we can write a sequence of instructions that perform a calculation. In software development, there are three types of control structures:

1. sequence: writing instructions one after the other
2. selection: either execute one set of instructions, or another
3. iteration: execute a set of instructions a finite or infinite number of times

As you already know how to sequence instructions one after the other, it is time to learn about selection and iteration. Let's start with selection:

```
if ( condition ) {
    instruction1;
    instruction2;
    //...
}
```

If statement

The instructions inside the if-branch are only executed if `condition` is truthy.

Notice the space between the `if` and the opening parentheses. This space is there to indicate that `if` is a *keyword* and not a function. Although this space is not mandatory, I highly recommend it for readability reasons.

```
function printNumber( x )
{

    if ( typeof x === 'number' )
    {
        console.log( x + ' is a number.' );
    }
}

printNumber(5);
console.log(printNumber());
printNumber(3);
```

If you run the code above, the first value is the console log. The second value is the return value of the `printNumber()` function. Remember, if the return value of a function is not specified, it returns `undefined`.

If we call the `printNumber` function with a non-numeric value, it does not print the console log, because the instructions inside the `if` branch are only executed, whenever the condition inside the `if` statement is truthy.

```
> printNumber( '' )
undefined
```

Adding another if statement

It is a bit awkward that we don't print anything in case the input is not a number. You already know everything to write a program that does this. The thought process is the following:

- if `x` is a number, print it
- if `x` is not a number, print the message `"The input has to be a number."`

```
function printNumber( x ) {
    if ( typeof x === 'number' ) {
        console.log( x + ' is a number.' );
    }
    if ( typeof x !== 'number' ) {
        console.log( 'The input has to be a number.' );
    }
}

printNumber(3);
printNumber("blah");
```

Else Statement

The second biggest problem with this solution is that we are writing too much for no reason.

The main problem with this solution is called *lack of abstraction*. Imagine that one day somebody comes and realizes that the condition `typeof x === 'number'` has to be modified. If you forget changing the second condition, you create an inconsistency in your code. We prefer thinking less when we don't have to. Therefore, I recommend another solution, where we only have one condition:

```
function printNumber( x ) {
    if ( typeof x === 'number' ) {
        console.log( x + ' is a number.' );
    } else {
        console.log( 'The input has to be a number.' );
    }
}

printNumber(3);
printNumber("blah");
```

Notice the `else` keyword. The `else` branch is only executed if the original condition is not true.

We can cascade this approach further, because after the `else`, you can have another condition:

```
function logLampColor( state ) {
    if ( state === 1 ) {
        console.log( 'Red' );
    } else if ( state === 2 ) {
        console.log( 'Yellow' );
    } else if ( state === 3 ) {
        console.log( 'Green' );
    } else {
        console.log( 'Wrong lamp state' );
    }
}
```

So, if the first condition fails, it will test the next `else if` condition, if that fails

So, if the first condition fails, it will test the next `else if` condition, if that fails too, then the next one and so on until all the `else if` fails and it will finally execute the `else` statement.

The beauty of the above code is that you can read it as if it was plain English. If the state is 1, then log Red. Otherwise if the state is 2, then log Yellow. And so on.

Switch statements

Another way of executing selective instructions is done through the switch statements.

There is an abbreviated version for a long chain of if-else statement: the `switch` operator. Switch is like a telephone operator. It puts you through some code in case the correct value is stored in your variable. There is also a default line, saying “the number you have dialed is invalid”.

```
function logLampColor( state ) {
  switch( state ) {
    case 1:
      console.log( 'Red' );
      break;
    case 2:
      console.log( 'Yellow' );
      break;
    case 3:
      console.log( 'Green' );
      break;
    default:
      console.log( 'Wrong lamp state' );
                }
}

logLampColor( 1 );
```

Try this code example out. Play around with it. Check what happens if you remove the `break` commands. Check what happens if you call this function with other arguments.

Now that you got a feel for the switch statement, let me explain it. You have a variable inside the switch. This variable may have values. We jump to the `case` label for which `state` is equal to the label value.

Then we start executing the code.

Why the `break` is so important?

If there was no `break` at the end of the code segment belonging to a label, execution would continue on the next code line. It does not matter that it is now under another label. Code just jumps through the labels.

This is why we have a `break` statement at the end of each code segment belonging to a label. The `break` statement jumps out of the closest switch statement, and your program continues execution after the `switch` statement.

We have two reasons not to include a `break` statement after a label:

- we use a `return` statement instead,
- we intentionally fall onto the next label (discouraged).

Example:

```
function getLampColor( state ) {
  switch( state ) {
    case 0:
    case 1:
      return 'Red';
    case 2:
      return 'Yellow';
    case 3:
      return 'Green';
                }
}
console.log(getLampColor( 0 ));
console.log(getLampColor( 1 ));
```

In this example, both cases `0` and `1` return `'Red'`.

The ternary operator

An overview of the ternary operator which also serves as a control structure

Notice you already know another way to return values without `if-else`:

```
function getLampColor( state ) {
    return state == 1 ? 'Red' : 'Green';
}

console.log(getLampColor(1));
console.log(getLampColor(5));
```

The ternary operator gives you red if the state is `1`, and green otherwise.

We can place another ternary expression in place of `'Green'` to add more lamp states:

```
function getLampColor( state ) {
    return state == 1 ? 'Red' :
           state == 2 ? 'Yellow' :
           state == 3 ? 'Green' :
           'Error';
}

console.log(getLampColor(1));
console.log(getLampColor(2));
console.log(getLampColor(3));
console.log(getLampColor(4));
```

Some people don't like this way of writing code. You can hit up their rant about how awful this construct is under the name of *ternary operator abuse*. I personally do not share their opinion, but this does not matter. If you believe you should use it, use it.

Q

```
var a = 3;
console.log( a%2 ? 'one' : 'zero' );
```

What will the output on console?

COMPLETED 0%

The While Loop

Learn how to repeat your code as many times as you want, by just writing down your code once. This lesson will introduce you to the while loop.

Introduction

Let's see iteration. In order to execute the same expressions multiple times, we create *loops*. A loop is like a cheap song. You know, Berlin is famous for the city of DJs. Sometimes I believe every third man claims he is a DJ. I happened to be a neighbor of one, fortunately, property management terminated his contract. This lunatic guy "worked" on one melody for half a year. And he repeated it over and over and over and over and over and over again.

Think of a loop like the song "Around the world, around the wo-orld. Around the world, around the wo-orld...". You get the idea. Take a melody, repeat it a hundred times, and you get the song. In programming terms, take some code, repeat it `x` amount of times, and you get code executed a hundred times.

Let's create a function that sums the values of an array:

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
```

The first four numbers are not by accident. They were the level selector cheat code of Sonic 2. If you have slight aspergers syndrome, you tend to remember some weird numbers.

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
function sumArray( values ) {
  let sum = 0;
  let i = 0;
  while ( i < values.length ) {
    sum += values[i];
    i += 1;
  }
  console.log( 'The loop was executed ' + i + ' times' );
  return sum;
}
```

```
sumArray( numbers );
```

The While Loop

The variable `i` keeps track of how many times the loop is executed.

First the JavaScript interpreter examines the condition in the `while` loop. As `i` is `0` and `numbers.length` is `13`, we conclude the condition is true.

If the condition is true, we execute the body of the while loop. So we add the ith element of the array to the sum, and increase the loop variable by `1`.

Then we compare `1` against the length of the array, `13`. As `1` is smaller than `13`, we execute the loop body again. The `sum` variable now stores `19 + 65 = 84`. The value of `i` becomes `2`.

We continue this until `i` becomes `13`. Then we realize the condition `i < numbers.length` becomes `false`. Once the condition of the `while` loop becomes `false`, we continue with the code after the `while` loop.

The following slides demonstrate four iterations of the while loop:

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 0

variable i: 0

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

TRUE

Sum: 0

variable i: 0

19	65	9	17	4	1	2	6	1	9	9	2	1

line executed

array element accessed

condition evaluated to


```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 19

variable i: 0

19	65	9	17	4	1	2	6	1	9	9	2	1

line executed

array element accessed

condition evaluated to

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 19

variable i: 1

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1


```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

TRUE

Sum: 19

variable i: 1

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 84

variable i: 1

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1


```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 84

variable i: 2

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

TRUE

Sum: 84

variable i: 2

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1


```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 93

variable i: 2

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 93

variable i: 3

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1


```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

TRUE

Sum: 93

variable i: 3

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1

```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 110

variable i: 3

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1


```
1  let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];
2
3  function sumArray( values ) {
4      let sum = 0;
5      let i = 0;
6      while ( i < values.length ) {
7          sum += values[i];
8          i += 1;
9      }
10     console.log( 'The loop was executed ' + i + ' times' );
11     return sum;
12 }
13
14 sumArray( numbers );
```

Sum: 110

variable i: 4

line executed

array element accessed

condition evaluated to

19	65	9	17	4	1	2	6	1	9	9	2	1

The Do-While Loop

You'll now learn about the do-while loop which is another way to repeat code.

We have a few more loops to go. The `do-while` loop executes the loop body **at least once**, and checks the condition before the end.

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];

function sumArray( values ) {
    if (  values.length == 0 ) return 0;
    let sum = 0;
    let i = 0;
    do {
        sum += values[i];
        i += 1;
    } while ( i < values.length );
    console.log( 'The loop was executed ' + i + ' times' );
    return sum;
}

sumArray( numbers );
```

Notice the first line in the function. We have to exit the `sumArray` function before reaching the `do-while` loop in case the array is empty. Otherwise, the code works in the exact same way as the while loop, except that the condition for running the loop again is at the bottom. If the condition is true, then it will repeat the loop as long as the condition is true.

Do-while for input validation

We mainly use do-while loops for input validation. The user inputs a value, then we check its validity.

Let's use the `prompt` function to get a value.

`prompt( 'message' )` opens a dialog displaying message. A textfield appears in

the dialog, where you can enter a value. Once you press the OK button, the `prompt` function returns a string.

Example:

Output

JavaScript

HTML

CSS (SCSS)

Run Code

This is where the do-while loop makes sense, because we can be sure we need to enter data at least once. In the example summing array members, using the do-while loop is technically possible, but it does not make sense, because the simple while loop is easier.

The For Loop

Get familiar with the For loop along with some optimization techniques regarding loops.

Introduction

You will now learn another loop that is perfect for iteration: the *for* loop. This loop combines several lines of code into one. The execution of the for loop is identical to the while loop:

```
initialization_of_loop_variable;
while ( condition ) {
    statements;
    increment_loop_variable;
}

for ( initialization_of_loop_variable; condition; increment_loop_variable ) {
    statements;
}
```

The for loop has everything you need in one line. You don't have to mix your loop body with the increment of the loop variable. The execution of the for loop is as follows:

1. `initialization_of_loop_variable`
2. check `condition`. If `condition` is falsy, exit the for loop
3. `statements` in the loop body
4. `increment_loop_variable`, then go back to step 2

Rewrite the `sumArray` function using a for loop. Here is the example with the `while` loop:

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];

function sumArray( values ) {
    let sum = 0;
    let i = 0;
    while ( i < values.length ) {
```

```
        sum += values[i];
        i += 1;
    }

    return sum;
}

sumArray( numbers );
```

Don't continue reading before you solve this exercise. Please change the while loop into a for loop.

I hope you liked the exercise. To verify your solution, check this code:

```
for ( let i = 0; i < values.length; i += 1 ) {
    sum += values[i];
}
```

This code should go in place of

```
let i = 0;
    while ( i < values.length ) {
        sum += values[i];
        i += 1;
    }
```

Notice you can declare variables in the initializer part of the for loop. The rest of the code works exactly like the `while` loop.

Let's simplify the for loop a bit further: you can write `i++` or `++i` in place of `i += 1`:

```
for ( let i = 0; i < values.length; ++i ) {
    sum += values[i];
}
```

Some optimization techniques

In some programming competitions, people tend to count downwards instead of upwards. The reason is that computing `values.length` costs some nanoseconds more than checking if the value of `i` is positive:

```
for ( let i = values.length - 1; i >= 0; --i ) {
    sum += values[i];
}
```

These wannabe geniuses don't stop there by the way. They go even further:

```
for ( let i = values.length; i; sum += values[--i] );
```

Weird, isn't it? Instead of iterating from `12` to `0`, we iterate from `13` to `1`. The condition `i` is `true` as long as its value stays non-zero. This is the easiest check a computer can make with an integer.

Suppose `i` is `13`. `values[-i]` is interpreted as `values[12]`, because first we decrease the value of `i`, then we equate the expression `-i` to the new value of `i`. This is a unique case when writing `i-` would have been incorrect, because our array does not even have an element at index `13`.

Don't worry if you don't feel like understanding this optimization technique. Once you read this paragraph for the tenth time, you will most likely get it. It's normal. Oh, and by the way, I discourage using such optimizations. Readability always trumps these slight edges. Sometimes you can save seconds in your code, so don't bother with the nanoseconds. It's like being penny-wise and pound-fool.

Another reason to avoid looking smart and optimizing your code is that JavaScript interpreters actually optimize code for you. This is a very easy and straightforward optimization. You don't have to bother doing it yourself. Just stick to this version of the for loop:

```
for ( let i = 0; i < values.length; ++i ) {
    sum += values[i];
}
```

Some more Loops

Other ways of writing loops are the For...in and For...of loop.

The for...in loop

Our next simplification is the `for…in` loop. Why bother iterating an `i` variable if a loop can take care of it for us?

```
for ( let i in values ) {
    sum += values[i];
}
```

The `for…in` loop enumerates all the available indices of the `values` array from `0` to `12` in ascending order.

The for...of loop

I have another simplification for you. What if I said, “why bother using a variable for iteration at all, if we could enumerate the values of the array?”. In ES6, there is a loop called the `for…of` loop, which does exactly that.

```
for ( let value of values ) {
    sum += value;
}
```

That’s all. Go back to the editor and substitute the for…in and the for…of constructs in place of your for loop. It works exactly the same way.

Break and Continue

In this lesson, you'll get to know about the break and continue keywords.

We are almost done. Just two small constructs.

Introduction

We can use `break` or `continue` inside the loops:

- `break` exits the closest loop it is in, and continues with the next command,
- `continue` skips out from the loop body, and jumps back to the condition of the loop.

Example: sum every second element of the `numbers` array:

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];

function sumArray( values ) {
    let sum = 0;
    for ( let i in values ) {
        if ( i % 2 == 0 ) continue;
        sum += values[i];
    }
    return sum;
}

console.log("The sum is", sumArray( numbers ));
```

Continue

This example is artificial, because we could have written `i += 2` in a simple for loop to jump to every second value. So we are testing `continue` just for the sake of the example. Whenever `i` is even, continue moves execution back to the next iteration of `i in values`.

Notice that we don't have to use braces around just one statement in the code if it is followed by an `if` statement or a loop:

```
if ( i % 2 == 0 ) continue;
```

is the same as

```
if ( i % 2 == 0 ) {
    continue;
}
```

We still prefer braces, because wrong code formatting may lead to many sources of error. For example, some people might think that `statement1` and `statement2` belongs to the loop body.

```
while ( condition )
    statement1;
    statement2;
```

Wrong!

If we added the braces, we would get the following code:

```
while ( condition ) {
    statement1;
}
statement2;
```

This must be very confusing for people familiar with Python. `statement2` is outside the loop.

Break

You already know what a `break` statement looks like, because we learned it when dealing with the `switch` statement. It is doing the same thing in loops. Suppose we want to break out from the loop whenever the sum exceeds 100:

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];

function sumArray( values ) {
    let sum = 0;
    for ( let i in values ) {
```

```
        sum += values[i];
        if ( sum >= 100 ) {
            break;

        }
    }
    return sum;
}

sumArray( numbers );
```

I placed the `break` in braces on purpose to show you that `break` breaks out of `switch` statements and loops, not from if statements. When `break` is executed, control is handed over to the statement after the loop: `return sum`.

In this specific example, we don't even need a `break`, because instead of breaking out of the loop, we can return the sum:

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];

function sumArray( values ) {
    let sum = 0;
    for ( let i in values ) {
        sum += values[i];
        if ( sum >= 100 ) return sum;
    }
    return sum;
}

sumArray( numbers );
```

Technically, this is an example, where I would suggest a more flexible for loop:

```
let numbers = [19, 65, 9, 17, 4, 1, 2, 6, 1, 9, 9, 2, 1];

function sumArray( values ) {
    let sum = 0;
    for ( let i = 0; i < values.length && sum <= 100; ++i ) {
        sum += values[i];
    }
    return sum;
}

sumArray( numbers );
```

Exercise 4

Create a random number using the expression

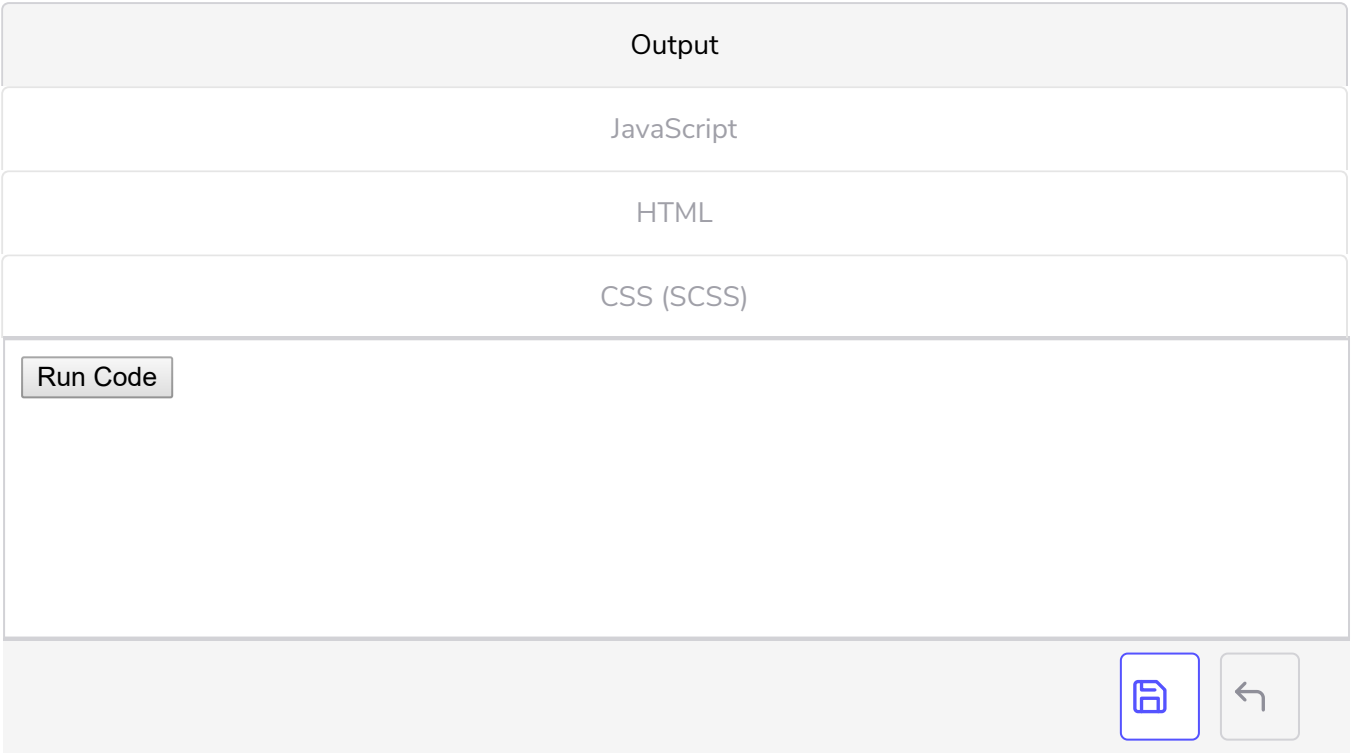
```
Math.trunc( Math.random() * 10 ) + 1
```

The above expression creates a random number between 1 and 10.

Create a function that asks the user for a number between 1 and 10 and repeat the question until a correct number is entered.

Help: prompt(‘Message’) gives you a popup displaying ‘Message’ and asks for an input. The return value of the prompt function is the input (string).

The solution is given below but don’t look at it until you put enough effort to solve the challenge.



Solution

Here is the solution.

```
let value = Math.trunc( Math.random() * 10 ) + 1

function guessValue( value ) {
    let guess;
    do {
        guess = prompt( 'Enter your guess (1 - 10):' );
    } while ( guess != value );
}

guessValue(value); //calling the function
```

You can copy paste it in the Javascript tab in the code editor above and run it. The program won't stop until you enter the correct number so keep trying by entering numbers from 1-10.

Using a do-while loop makes sense, because we have to get a guess first to be able to evaluate the condition guess != value. Note that the main use case for do-while loops is to validate user input.

Let's add some more functionality

Your program should now also print one of the following messages once a value is entered:

- "The value you entered is too high! Try entering a lower number!"
- "The value you entered is too low! Try entering a higher number!"
- "Correct! 36 is the right answer." - here, assume 36 was the number you had to guess
- "Invalid guess! You have to enter a number between 1 and 50."
- "Invalid format! Only numbers are allowed!"

Hint: You can use the `alert` function to print messages. The `alert("message here")` method displays a pop-up box with a specified message and an OK button.

Output

JavaScript

HTML

CSS (SCSS)

Run Code

Solution

```
function logHintOnGuess( guess, value ) {
    guess = Number.parseInt( guess, 10 );
    if ( Number.isNaN( guess ) ) {
        alert( "Invalid format! Only numbers are allowed!" );
    } else if ( guess < 1 || guess > 10) {
        alert( "Invalid guess! You have to enter a number between 1 and 1
0." );
    } else if ( guess > value ) {
        alert( "The value you entered is too high! Try entering a lower nu
mber!" );
    } else if ( guess < value ) {
        alert( "The value you entered is too low! Try entering a higher nu
mber!" );
    } else if ( guess == value ) {
        alert( "Correct! " + value + " is the right answer." );
    }
}
let value = Math.trunc( Math.random() * 10 ) + 1

function guessValue( value ) {
    let guess;
    do {
        guess = prompt( 'Enter your guess (1 - 10):');
        logHintOnGuess( guess, value ); //calling the Hint function here
    } while ( guess != value );
}

guessValue(value); //calling the function
```

You can copy paste it in the Javascript tab in the code editor above and run it.

You already know everything to read this function. I will still explain the thought process behind writing it.

- First of all, we have to convert the guess to an integer to see if the format is correct.
- We have to swap the order of the messages when checking them in the if-else if-else structure. Because when the guess is not a number, we have to display an invalid format message.
- Then we check if our guess is outside the specified boundaries. In general, checking the error cases first ensures that after all the errors are covered, we can assume that the input is correct
- The order of the low guess, high guess, and exact guess message does not matter. However, notice that in the very last else branch, the presence of the condition is optional. We could comment out the guess == value condition.

Value and Reference Types

An entry-level introduction to value and reference types in JavaScript.

Introduction

JavaScript does not give you full access to your data structures in memory. However, reference types still exist in the language. Mixing value and reference types comes with unwanted side-effects and bugs. Understanding the difference between value and reference types plays a vital role in writing robust programs.

> Numbers, booleans, strings, null and undefined are primitive types. All primitive types are passed by value. Objects, arrays, and functions are passed by reference.

Strings are somewhat special in JavaScript. Unlike many other languages, strings are not defined as arrays of characters. What is more, the character type does not even exist in JavaScript. Strings are immutable data types, with an interface that makes them look like an array of characters. In practice, any modifications to a string result in a new, immutable string value.

After the introduction of value and reference types, let's continue with a riddle. Determine the result of the console logs.

```
var people = [
    { name: 'John Hill', age: 22 },
    { name: 'Jack Chill', age: 27 }
];

var getInitials = function( name ) {
    // Reusing the name argument makes little sense in general.
    // We are making this assignment here for demonstrating
    // the difference between value types and reference types.
    name = name.split( ' ' ).map( function( word ) {
        return word.charAt( 0 );
    } ).join( '' );
```

```
    console.log( name );

    return name;
}

var increaseAge = function( person ) {
    person.age += 1;
}

var addPerson = function( people, name, age ) {
    people.push( { name: name, age: age } );

}

// Part 1: getInitials
console.log( getInitials( people[0].name ) );
console.log( people[0].name );

// Part 2: increaseAge
increaseAge( people[1] );
console.log( people[1].age );

// Part 3: addPerson
addPerson( people, 'Jim Gordon', 32 );
console.log( people );
```

Question:

```
What is logged to the console?
```

Answer:

- In the first part, the initials of a name are constructed by the `getInitials` function. The string value passed to the function is copied. This copy is accessed and modified inside the function. These modifications have nothing to do with the original value `people[0].name`.
- The `increaseAge` function accepts an object. As objects are reference types, only the reference to the passed `person` is copied. When members of the referenced object are changed, the changes are kept even after executing the function. These changes are accessible via each reference.
- Arrays are also reference types. Adding an element to an array is a permanent change.

```
JH
JH
John Hill
```

```
28
(index) name         age
0       "John Hill"  22
1       "Jack Chill" 28
2       "Jim Gordon" 32
```

Mistake: treating a value type as a reference type

This lesson will show two erroneous attempts of assuming a value type as a reference type.

Suppose that the `increaseAge` method was erroneously implemented like this:

```
var people = [
    { name: 'John Hill', age: 22 },
    { name: 'Jack Chill', age: 27 }
];

function increaseAge( age ) {
    age += 1;
}

console.log( 'Before:', people[0].age );
increaseAge( people[0].age );
console.log( 'After:', people[0].age );
```

The age stays `22` even after executing the `increaseAge` function. Primitive types are passed by value. Changing the copied value has no effects on the original value. Always encapsulate the fields you would like to change with an array or an object.

Another example is an erroneous attempt to replace a person with another one. Suppose that the `replacePerson` function accepts 3 arguments: the person to be replaced, and the name and age of the new person.

```
var people = [
    { name: 'John Hill', age: 22 },
    { name: 'Jack Chill', age: 27 }
];

function replacePerson( person, name, age ) {
    person = { name: name, age: age };
}

replacePerson( people[1], 'Jack Newtown', 35 );
console.log( people );
```

This function does not do anything to the `people` object as the `person` argument was assigned a new value. The `people` data structure stays intact. The `person` handle inside the function references a completely new object after the assignment. Once this function terminates, the new object is thrown away.

Mistake: treating a reference type as a value type

This lesson will show you how to fix a problem of assuming a reference type as a value type.

Suppose that a customer enters a supermarket and buys a chewing gum. The customer was also a diligent collector of empty bottles and happened to cash in 4000 bottles in exchange for a 1000€ voucher. The cashier is checking if there are enough funds to accept the payment. This check is performed by the `canChange` function returning a boolean result:

```
//shopTransaction for chewing gum
var shopTransaction = {
    items: [ { name: 'Astro Mint Chewing Gum' } ],
    price: 1,
    amountPaid: 1000
}

var cashier = {
    units:    [500, 200, 100, 50, 20, 10, 5, 2, 1],
    quantity: [0, 0, 5, 4, 5, 10, 10, 20, 9]
}

function canChange( shopTransaction, cashier )
{    //amount that has to be paid
    var amount = shopTransaction.amountPaid - shopTransaction.price;
  //checking if the amount can be paid after using
  //every quantity of a unit
  for( var i = 0; i < cashier.units.length; ++i )
    {
        var unit = cashier.units[i];
        while ( amount >= unit && cashier.quantity[i] > 0 )
        {
            amount -= unit;
            cashier.quantity[i] -= 1;
        }
    }

    return amount == 0;
}

console.log( canChange( shopTransaction, cashier ) );
console.log( cashier.quantity );
```

Although the function returns the value we expect, there is still a problem. The side effect of executing the `canChange` function is that all the notes disappeared from the cashier. Why?

The problem is that `cashier` is a reference type. All changes to fields inside the cashier are preserved even after exiting the function.

A possible solution is to create a new variable and equate it to `cashier.quantity[i]`. This will copy the value of `cashier.quantity[i]` to a new location in memory as we are dealing with a primitive type.

```
function canChange( shopTransaction, cashier ) {
    var amount = shopTransaction.amountPaid - shopTransaction.price;
    for( var i = 0; i < cashier.units.length; ++i ) {
        var unit = cashier.units[i];
        var currentQuantity = cashier.quantity[i];
        while ( amount >= unit && currentQuantity > 0 ) {
            amount -= unit;
            currentQuantity -= 1;
        }
    }

    return amount == 0;
}
```

In theory, it is also possible to deeply clone all reference types in functions that are not supposed to mutate their values. Deep cloning is easier said than done. We will explore the different types of cloning in the next chapter.

Shallow Copy

An introduction to cloning objects and shallow copy

Introduction

JavaScript made its way from being a toy language for simple animations to becoming a language for client side and server side web application development. Some generic concepts also made their way to the JavaScript world and developers become more and more aware of them. Cloning objects is one of these concepts.

Cloning can be quite complex. Prototypal inheritance, reference types and methods associated with an object may require a specialized approach. Restrictions on cloned data may simplify cloning. It is the responsibility of the developers to understand and apply the correct cloning method on a case by case basis.

Cloning methods of most libraries are implemented using shallow copying. One example is `_.clone`, the clone method of UnderscoreJs.

Shallow Copy :
All field keys and values of the original object are copied to the new object.

This definition has some implications. Recall the `shopTransaction` object from the last chapter.

```
var shopTransaction = {
    items: [ { name: 'Astro Mint Chewing Gum' } ],
    price: 1,
    amountPaid: 1000
}

var clonedTransaction = _.clone( shopTransaction );

clonedTransaction.price = 3;
clonedTransaction.items.push( { name: 'Tom&Berry Frozen Yoghurt' } );

console.log( 'clonedTransaction.price = ', clonedTransaction.price );
console.log( 'shopTransaction.price = ', shopTransaction.price );
```

```
console.log( 'clonedTransaction.items.length = ', clonedTransaction.items.length );

console.log( 'shopTransaction.items.length = ', shopTransaction.items.length );
```

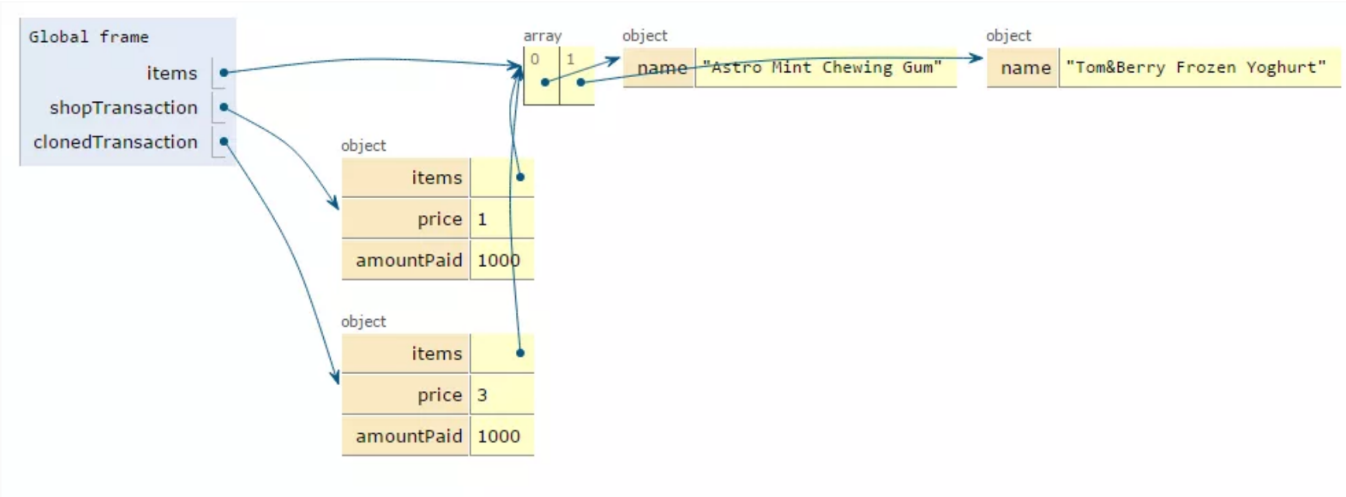
Both `clonedTransaction` and `shopTransaction` have the same properties. As `price` and `amountPaid` are value types (numbers), their values are copied while cloning. The `items` property is a reference type. Therefore, both shopTransaction and clonedTransaction point to the exact same array in memory.

Changing the value of primitive types in `clonedTransaction` has no effects on the original object. However, `shopTransaction.items` and `clonedTransaction.items` are references pointing to the exact same array. It does not matter which reference we use to add an element to the array, the results are going to be visible under both references. Therefore, the result of the execution of the above code is:

```
clonedTransaction.price =  3
shopTransaction.price =  1
clonedTransaction.items.length =  2
shopTransaction.items.length =  2
```

> Modifying a value through a reference inside a cloned object also modifies the original object.

Check out a visual representation of the objects below.



Prototypal Inheritance

An introduction to Prototypal Inheritance

The extend method of UnderscoreJs adds all properties inherited via the prototype as own properties.

```
var proto = { protoProperty: 'proto' };
var o = Object.create( proto );

var c = _.extend({}, o);

console.log( 'o.hasOwnProperty( "protoProperty" )', o.hasOwnProperty( "protoProperty" ) );
// o.hasOwnProperty( "protoProperty" ) false

console.log( 'c.hasOwnProperty( "protoProperty" )', c.hasOwnProperty( "protoProperty" ) );
// c.hasOwnProperty( "protoProperty" ) true
```

While some shallow copy implementations behave in the same way, others completely omit properties coming from the prototype chain.

More often than not, the prototype should not be considered at all. In case of cloning objects with a prototype, make sure you consider how the clone method treats prototypes.

Are Shallow Copies Dangerous?

Let's assess if using shallow copies is safe or not.

We can conclude that making shallow copies are sometimes not enough. When the copied object is modified, it may result in unwanted consequences. Most of the time, software development best practices and restricted workflow allow us to use shallow copies of objects without problems.

For instance, in BackboneJs, the default implementation of the `toJSON` method of models looks like this:

```
toJSON: function(options) {
    return _.clone(this.attributes);
}
```

I have not seen complaints on BackboneJs forums addressing the problem of retrieving the results of the `toJSON` method and modifying the contents of the model attributes. The reason is that the `toJSON` method is mostly used for one of the two purposes:

- Serialization: preparing a JSON payload for an AJAX request,
- Presentation: preparing a JavaScript object and giving it to a templating engine.

Although some extreme cases may apply, in general, both of these operations only access the clone. Modifying nested data structures are hardly required.

It is very important to detect situations when shallow copies are not enough. In these cases, either think about refactoring your code or using a deep copy.

Deep Copy

An introduction to deep copy

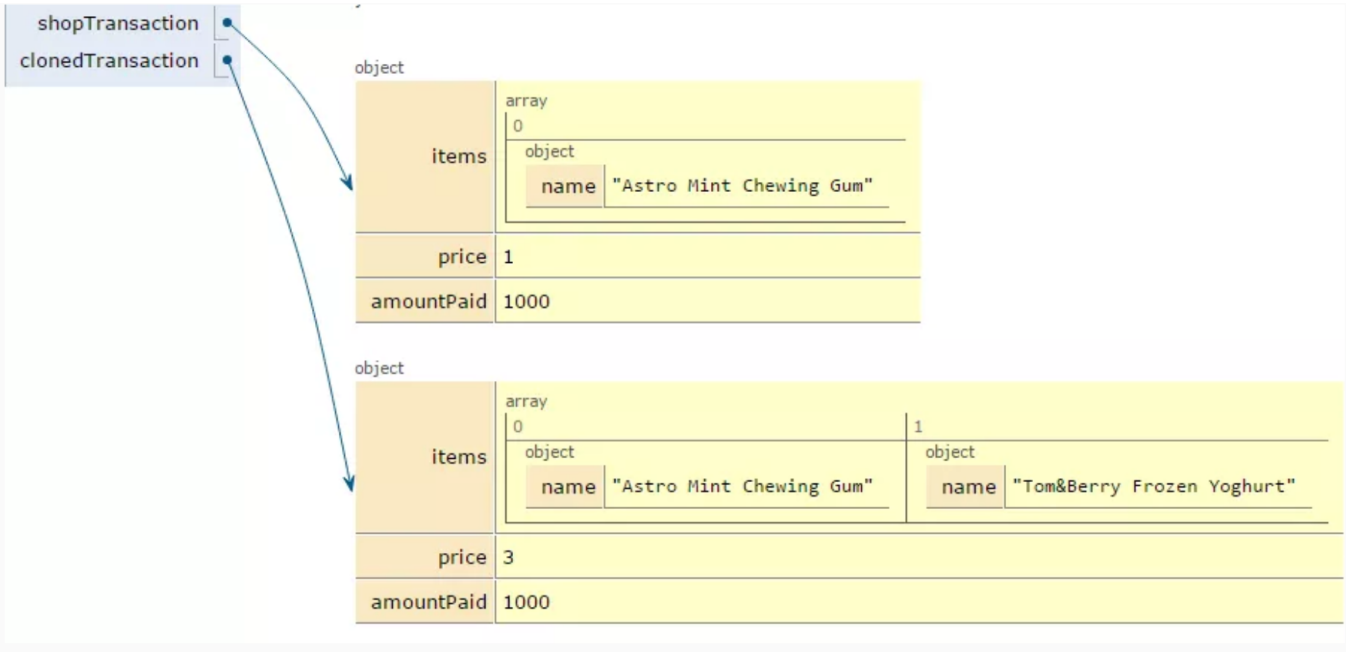
When deeply cloning an object, all references are dereferenced. Only the structure of the object, key names and the atomic values are kept. A deep copy requires traversal of the whole object and building the cloned object from scratch.

```
var shopTransaction = {
    items: [ { name: 'Astro Mint Chewing Gum' } ],
    price: 1,
    amountPaid: 1000
}

var clonedTransaction = deepCopy( shopTransaction );

clonedTransaction.price = 3;
clonedTransaction.items.push( { name: 'Tom&Berry Frozen Yoghurt' } );

console.log( 'clonedTransaction.price = ', clonedTransaction.price );
console.log( 'shopTransaction.price = ', shopTransaction.price );

console.log( 'clonedTransaction.items.length = ', clonedTransaction.items.length );
console.log( 'shopTransaction.items.length = ', shopTransaction.items.length );
```

The deep copy made it possible to modify the `items` member of both transaction objects individually.

```
clonedTransaction.price =  3
shopTransaction.price =  1
clonedTransaction.items.length =  2
shopTransaction.items.length =  1
```

The image below shows that `shopTransaction` and `clonedTransaction` are fully distinct: no values are reachable from both objects.



Another necessary condition for a sound deep clone function is cycle detection. Cloning functions should terminate even if an object references itself. It is possible to clone these objects in case a lookup table of the references are constructed.

Many cloning methods based on an intermediate representation fail though as the intermediate representation has to be finite.

Restricted Deep Copy Implementations

This lesson will teach two methods to implement deep copy and their limitations.

JSON methods

There is a very easy implementation for making deep copies of JavaScript objects. Convert the JavaScript object into a JSON string, then convert it back into a JavaScript object.

```
var deepCopy = function( o ) {
    return JSON.parse(JSON.stringify( o ));
}
```

Restrictions:

- Object `o` has to be finite, otherwise `JSON.stringify` throws an error.
- The `JSON.stringify` conversion has to be lossless. Therefore, methods are not allowed as members of type `function` are ignored by the JSON stringifier. The `undefined` value is not allowed either. Object keys with `undefined` value are omitted, while undefined values in arrays are substituted by `null`.
- Language hacks won't work. Associative properties of an array will not appear in the cloned object. Example: `a = []; a.b = language hack';`. The value `a.b` will be accessible in the original object, but it will disappear from the deep copy.
- The prototype of the copy becomes `Object`. All properties coming from the prototype chain will be discarded.
- Members of the Date object become ISO-8601 strings, not representing the timezone of the client. The value `new Date(2011,0,1)` becomes `"2010-12-31T23:00:00.000Z"` after executing the above deep copy method in case you live in Central Europe.

I have hardly ever found any problems with these restrictions when using JSON methods to implement the deep copy functionality. It is still important to know what can potentially go wrong.

jQuery Extend

```
var deepCopy = function( o ) {
    return $.extend( true, {}, o );
}

var shallowCopy = function( o ) {
    return $.extend( {}, o );
}
```

When the first argument of `$.extend` is `true`, it performs a deep extension. Deeply extending the empty object is the same as cloning the object.

Restrictions:

- Object o has to be finite, otherwise `$.extend` throws a `RangeError` for exceeding the maximum stack size
- The prototype of the copy becomes `Object`. All properties coming from the prototype chain will be added as own properties

There are multiple implementations for performing a deep clone. Most of them are restricted in some way. Relying on a generic deep cloning method is hardly ever needed. Whenever an object becomes too complex to clone, it is a good sign that the code has to be refactored.

Know what you're cloning

Cloning in JavaScript is not straightforward. As there is no native implementation for cloning, we have to implement it ourselves or we have to rely on a third party solution. These solutions work perfectly under some restrictions. Be aware of whether you perform deep or shallow cloning and also consider the restrictions. Whenever simple cloning solutions don't work, more often than not, think about refactoring your code instead of looking for a more reliable cloning function.

Javascript Strings

Lets start with how to create strings and check their equality.

In software development, working with strings is a common problem. We often read, process, and write text files, perform logging on the activities of a system, or analyze user input.

Learning how to perform string operations is essential in any programming language.

Creating JavaScript strings

Strings contain a sequence of characters. You can either use single quotes (e.g. `'Javascript in Practice'`) or double quotes (e.g. `"Javascript in Practice"`) to create strings.

```
let firstString  = "Hugo's dog",
    secondString = 'Hugo\'s dog';
```

In the console, strings are displayed with double quotes.

As you can see in the previous example, it is possible to use a single quote inside a double quote, and it is also possible to use a double code inside a single quote. If you want to use a single quote inside a single quote, you have to escape it with a backslash.

The `\` tells the JavaScript interpreter that the next character should be treated as a literal character, not as a meta character. Therefore, `\'` does not signal the start or the end of a string.

Side note: if you want to place a backslash in a string, you have to escape it in the form of | \|.

Equality of strings

Two strings are equal when exactly their values are equal:

Two strings are equal whenever their values are equal:

```
let firstString  = "Hugo's dog",
    secondString = 'Hugo\'s dog';
console.log(firstString == secondString);
//true
console.log(firstString === secondString)
//true
console.log(firstString === 'Hugo\'s dog');
//true
```

Sorting Strings

This lesson will introduce you to the localCompare method which helps us to sort strings.

Often in software development, we have to sort strings. There are a few problems with string sorting:

- Uppercase and lowercase letters are sorted differently: `'a' > 'B'`,
- accented characters are completely out of sequence: `'á' > 'b'`.

The localCompare method

The localCompare string method compares two string in the current locale which is based on the language settings of your browser. It helps solves both the above problems:

```
console.log('á'.localeCompare( 'b' ));
console.log('á'.localeCompare( 'a' ));
console.log('a'.localeCompare( 'A' ));
console.log('b'.localeCompare( 'A' ));
console.log('A'.localeCompare( 'b' ));
console.log('B'.localeCompare( 'b' ));
```

Sorting an array of strings in place works as follows:

```
const words = [ 'Practice', 'ES6', 'in', 'á' ];
const sorter = function( a, b ) {
    return a.localeCompare( b );
}

words.sort( sorter );
console.log(words);
```

This sort method of arrays expects a helper function such as `sorter`. This

This sort method of arrays expects a helper function such as `sorter`. This helper function expects two arguments, `a` and `b`. The helper function should

be written in such a way that it should return a positive value whenever `a > b`, a negative value whenever `a < b`, and zero if `a` and `b` are equal.

The `sort` JavaScript array method sorts its contents in place. This means the order of the elements change inside the array.

Length of a String And Inserting New Lines

This lesson will teach you to get the length of a string and to insert a new line in a string.

The length of a string

Strings have a `length` property that is equal to the number of characters in the string. The shortest possible string is the *empty string* having a length of `0`:

```
let firstString  = "Hugo's dog";

console.log(firstString.length)
console.log('').length)
```

Multiline strings

When defining the string, the start and end of the string has to be in the same line. Therefore, we need to insert a *newline character* into the string to make it multiline:

```
console.log('First line.\nSecond line.\nThird line.');
```

Alternatively, you can place a backslash at the end of the line:

```
'First line.\
Second line.\
Third line.'
```

Both solutions are inconvenient. Fortunately, in ES6, template literals were introduced which we have covered in the 'JavaScript in Practice: ES6, ES2017

and beyond’ course.

Trimming Strings

An introduction to the trim method

Trimming is the process of removing whitespace characters before the first non-whitespace character and after the last non-whitespace character of the string.

We often remove these unwanted whitespace characters when processing templates or processing user input. For instance, assuming that `_` denotes a space, instead of entering `_Zsolt` in a form field, you would expect that `Zsolt` is saved in the database.

The `trim` string method performs trimming:

```
const template = `
<div>First line</div>
<div>Second line</div>
`;
console.log(template);
console.log("After trimming:");
console.log(template.trim());
```

Accessing Characters inside a String

This lesson will introduce you to ways to access characters inside a string. Moreover, it will also teach you to modify characters in a string.

The Bracket Notation

The bracket notation, also used with arrays, can provide access to an arbitrary character in a string:

```
let digits = '0123456789';
console.log(digits[4]);
```

The `charAt` Method

Instead of indexing, you can also use the `charAt` method:

```
let digits = '0123456789';
console.log(digits.charAt(4));
```

Opposed to arrays, setting a character inside the string to a new value doesn't work. Indexing a string is strictly *read-only*:

```
let digits = '0123456789';
digits[4] = 'X';
console.log(digits)
```

Setting `digits[4]` to `'X'` failed silently.

In general, strings are said to be *immutable*. This means that we cannot change their content. When adding a character to the end of the string, a new string is created.

To iterate on a string, all JavaScript control structures can be used: `for`, `while`, `do...while`, `for...in`, `for...of`:

```
let digits = '0123456789';
let sum1 = "";
for ( let i = 0; i < digits.length; ++i ) {
    sum1 += digits[i];
}

let sum2 = "";
for ( let i in digits ) {
    sum2 += digits[i];
}

let sum3 = "";
for ( let digit of digits ) {
    sum3 += digit;
}

console.log(sum1);
console.log(sum2);
console.log(sum3);
```

Accessing Characters inside a String

This lesson will introduce you to ways to access characters inside a string. Moreover, it will also teach you to modify characters in a string.

The Bracket Notation

The bracket notation, also used with arrays, can provide access to an arbitrary character in a string:

```
let digits = '0123456789';
console.log(digits[4]);
```

The `charAt` Method

Instead of indexing, you can also use the `charAt` method:

```
let digits = '0123456789';
console.log(digits.charAt(4));
```

Opposed to arrays, setting a character inside the string to a new value doesn't work. Indexing a string is strictly *read-only*:

```
let digits = '0123456789';
digits[4] = 'X';
console.log(digits)
```

Setting `digits[4]` to `'X'` failed silently.

In general, strings are said to be *immutable*. This means that we cannot change their content. When adding a character to the end of the string, a new string is created.

To iterate on a string, all JavaScript control structures can be used: `for`, `while`, `do...while`, `for...in`, `for...of`:

```
let digits = '0123456789';
let sum1 = "";
for ( let i = 0; i < digits.length; ++i ) {
    sum1 += digits[i];
}

let sum2 = "";
for ( let i in digits ) {
    sum2 += digits[i];
}

let sum3 = "";
for ( let digit of digits ) {
    sum3 += digit;
}

console.log(sum1);
console.log(sum2);
console.log(sum3);
```

Finding the Index of a Substring inside a String

An introduction to the indexOf and lastIndexOf methods which help us to find the index of a specified substring in a string

The `indexOf` and the `lastIndexOf` string methods return the first and last index of a substring inside a string.

```
let print;
let sequence = '1,2,3,4,5';
print = sequence.indexOf( ',' )
console.log(print);
//1
print = sequence.lastIndexOf( ',' )
console.log(print);
//7
print = sequence.indexOf( ',3' )
console.log(print);
//3
print = sequence[3]
console.log(print);
//","
```

When the argument of `indexOf` is a string of length higher than `1`, the return value is the position of the first character.

At this point, we assume that you don't use *long unicode characters*. As soon as you know you will use these characters, check out my article Strings and Template Literals in ES6 to know how to deal with them.

What if the string does not contain the specified substring?

When string `s` does not contain a specific substring `s0`, then `s.indexOf( s0 )` returns `-1`:

```
let print;
let sequence = '1,2,3,4,5';
print = sequence.indexOf( 'abc' )
```

```
console.log(print)
//-1
```

Assuming you want to enumerate the indices of all matches, you can specify a second argument, indicating the first index of the string from where we start searching:

```
let print;
let sequence = '1,2,3,4,5';
print = sequence.indexOf( ',' )
console.log(print)
//1
print = sequence.indexOf( ',', 1 + 1 )
console.log(print)
//3
print = sequence.indexOf( ',', 3 + 1 )
console.log(print)
//5
print = sequence.indexOf( ',', 5 + 1 )
console.log(print)
//7
print = sequence.indexOf( ',', 7 + 1 )
console.log(print)
//-1
```

The question “Does string `s` include the substring `s0`?” is commonly asked during programming problems. We could use `indexOf` to implement the answer:

```
let print;
let sequence = '1,2,3,4,5';
let s0 = '8,9'
print = sequence.indexOf(s0) >= 0
console.log(print)
```

Splitting, Slicing, Joining, and Concatenating Strings

A list of the methods that you will learn in future chapters

Strings have properties. One example is the `length` property.

There are also some operations defined on strings. These operations are called methods. Some of them are listed below:

- `.split()`
- `.slice()`
- `.join()`
- `.replace()`

We learned in one of the previous lessons that `digits[4] = 'X'` does not set a character inside the digits string.

By the end of this chapter, we will know everything needed to change a character inside strings.

Splitting a String into an Array of Substrings

In this lesson, we will split strings into arrays using the 'split' method.

The Split Method

The `split` method splits a string into an array of substrings. Split expects one argument describing how the split should be made.

For instance, in the programming world, we often process CSV files. CSV stands for Comma Separated Values. Let's create an array from a CSV line:

```
const line = '19,65,9,17,4,1,2,6';
console.log(line.split( ',' ));//console.log for printing
//["19", "65", "9", "17", "4", "1", "2", "6"]
```

If we want to create an array containing each character in the string, we can pass an empty string to the `split` method:

```
const line = '19,65,9,17,4,1,2,6';
console.log(line.split( '' ));
//["1", "9", ",", "6", "5", ",", "9", ",", "1", "7", ",", "4", ",", "1", ",", "2", ",", "6"]
```

As the contents of arrays can be changed, we can easily change the digit `4` inside the `digits` sequence:

```
let digitsArray = '0123456789'.split( '' );
digitsArray[4] = 'X';
console.log(digitsArray)
//["0", "1", "2", "3", "X", "5", "6", "7", "8", "9"]
```

Splitting using Regular Expression

Split also works with a regular expression argument. In this case, the regex describes the pattern used for splitting strings. For instance, `/\D/` matches every character that is not a digit. Splitting based on this regular expression works as follows:

```
console.log('0,1,2,3,4,5,6,7,8,9'.split( /\D/ ));
//["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]
```

If you want to learn more about regular expressions, check out my JavaScript Regex Udemy video course. Use the coupon `JSREGEX` to get it at minimum price.

Joining Strings

This lesson will introduce you to the join method.

We can join strings in two ways:

- using the `+` operator,
- using the `join` string method.

You already saw how the `+` operator works on strings in the introductory chapters:

```
let print = 'first part' + ' second part'
console.log(print);
//"first part second part"
```

The Join Method

The `'join'` method is a bit more interesting. It takes an array of strings and joins them into one string. When `join` is used without any arguments, it joins the strings by inserting commas in-between them. This is because the most common string writing functionality is logging and creating CSV files.

```
console.log(["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"].join());
//"0,1,2,3,4,5,6,7,8,9"
```

Joining without Commas

If you want to join strings without anything in-between them, specify an empty string as the argument of `join`:

```
console.log(["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"].join( '' ))
//"0123456789"
```

Joining with other Characters

You can also specify any other characters:

```
console.log(["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"].join( '👍👍' ));
//"0👍👍1👍👍2👍👍3👍👍4👍👍5👍👍6👍👍7👍👍8👍👍9"
```

Slicing A String

An introduction to the slice method

The Slice Method

The `slice` method is commonly used in simple functions and coding challenges. It returns a substring of the string specified by its argument list.

> The easiest way to remember the parameter list of `slice` is:
>
> `str.slice(firstIndexInString)`
>
> or
>
> `str.slice(firstIndexInString, firstIndexAfterString)`

The first argument of `slice` specifies the position of the first character of the substring. The second argument is optional. When it is missing, slicing happens until the end of the string. When it is specified, it points at the first character after the sliced substring.

```
let hexadecimalDigits = '0123456789ABCDEF';
console.log(hexadecimalDigits.slice( 1, 6 ))
//"12345"
console.log(hexadecimalDigits.slice( 10 ))
//"ABCDEF"
console.log(hexadecimalDigits.slice( 0, 10 ))
//"0123456789"
```

Slice Using Negative Values

Similar to Python arrays, the arguments of `slice` can be negative. Negative

values count from the end of the array:

```
let hexadecimalDigits = '0123456789ABCDEF';
console.log(hexadecimalDigits.slice( -6 ))
//"ABCDEF"
console.log(hexadecimalDigits.slice( -6, -3 ))
//"ABC"
console.log(hexadecimalDigits.slice( -6, 13 ))
//"ABC"
```

Final Notes

You could learn the `substr` and `substring` methods that perform slicing using a different syntax. However, you will end up using `slice` most of the time anyway, therefore, in this summary, we will omit these two methods. Substr works like slice, but it allows the first argument to be greater than the second one, and still return the substring between the indices. The `substring` method specifies the index of the first character and the length of the substring.

The reason why I advise only using `slice` is that you will use the same method with arrays too. Its parametrization is intuitive, and you can also understand Python array slicing better.

Replacing Characters of a String

This lesson will teach you a number of methods to replace characters in a string.

The Replace Method

The `replace` string method returns a new string, where the first substring specified by its first argument is replaced with its second argument:

```
const numbers = '1 2 3 4';
console.log(numbers.replace( ' ', ',' ));
//"1,2 3 4"
```

Notice that only the first space was replaced. If you want to replace all spaces inside a string, you can use the `split` and `join` methods:

```
const numbers = '1 2 3 4';
console.log(numbers.split( ' ' ).join( ',' ))
//"1,2,3,4"
```

Replacing by using Regular Expression

Alternatively, you can also specify a regular expression as the first argument of the `replace` method, and apply a global flag on it to replace all matches.

```
const numbers = '1 2 3 4';
console.log(numbers.replace( / /g, ',' ))
//"1,2,3,4"
```

This solution is a bit advanced. Head over to my article Regular Expressions in JavaScript if you want to learn more.

You can replace any number of characters including zero. In case of replacing the empty string, the second argument is inserted before the first character:

```
console.log('help'.replace( '', '--' ))
//"--help"
console.log('help'.replace( new RegExp( '', 'g' ), '--' ))
//"--h--e--l--p--"
console.log('1 2 3 4'.replace( '2 3', 'five' ))
//"1 five 4"
```

Upper and Lower Case Letters

This lesson will teach you to covert lower-case letters to upper-case letters and vice versa.

The `toUpperCase` and `toLowerCase` string methods return an upper case and a lower case version of their string respectively:

```
console.log('aBCd'.toLowerCase())
//"abcd"
console.log('aBCd'.toUpperCase())
//"ABCD"
```

Exercise on String Manipulation

Further your knowledge of string manipulation using Javascript string functions.

There is a typo in the string `word`, symbolized by the `X` character inside the string. Correct it by reusing all other characters of the original string, and place the result in the `word` variable.

```
let sentence = 'This sentence contains one spelling erXor.';
console.log(sentence);
```

Solution

a) We will make the assumption that in this string, 'X' does not occur anywhere else in the string. If this assumption did not hold, we would be able to look up 'erXor' in the same way to replace an erroneous word instead of an erroneous letter.

All we need to do is split the string into two substrings. One ending right before 'X', and one starting right after 'X'. Then we can join the two strings and place an 'r' between them.

```
let sentence = 'This sentence contains one spelling erXor.';
console.log(sentence);
sentence = sentence.split('X').join('r')
console.log(sentence);
```

b) We can parametrize slice by counting from the end of the string. We need the string except for the last 4 character, one 'r' character, and the last 3 characters. The '+' operator concatenates these substrings:

```
let sentence = 'This sentence contains one spelling erXor.';
console.log(sentence);

sentence = sentence.slice( 0, -4 ) + 'r' + sentence.slice( -3 )
console.log(sentence);
```

Alternatively, you can also use join to join the substrings:

```
let sentence = 'This sentence contains one spelling erXor.';
console.log(sentence);
sentence = [ sentence.slice( 0, -4 ), sentence.slice( -3 ) ].join( 'r' )
console.log(sentence);
```

Exercise on Join, Map and Split

Improve your knowledge of the join, map and split functions by solving this exercise.

After correcting the spelling error in the previous exercise, capitalize each word of the sentence. Each word should start with an uppercase letter, while the rest of the letters should be in lower case.

```
let sentence = 'This sentence contains one spelling error.';
console.log(sentence);
```

Solution:

First, we split the sentence into an array of words. Then we slice each word such that we separate their first character and the rest. The third step is to capitalize the first letters with `toUpperCase`. Then we join everything.

```
let sentence = 'This sentence contains one spelling error.';
const words = sentence.split( ' ' );
for ( let i = 0; i < words.length; ++i ) {
    words[i] = words[i].slice(0, 1).toUpperCase() + words[i].slice( 1 );
}

let newsentence=words.join(' ');
console.log(newsentence);
```

Note: an alternative for `words[i].slice( 0, 1 )` is `words[i].charAt( 0 )`.

If you prefer writing the solution in a more concise way, it is possible to use an array helper function called map. Map executes a function on each element of the array and returns an array with the transformed elements. The order of the elements is kept. Example:

```
let numbers = [1, 2, 3];
const tripleEachElement = function( x ) { return 3*x }
// or using the arrow syntax: const triple = x => 3*x;

numbers = numbers.map( tripleEachElement )
console.log( numbers )
```

Let's use the mapping idea to implement a solution:

```
let sentence = 'This sentence contains one spelling error.';
let words = sentence.split( ' ' );

let capitalize = function( word ) {
    return word.charAt(0).toUpperCase() + word.slice(1);
}

words = words
    .map( capitalize )
    .join( ' ' );

console.log(words);
```

No wonder why functional programming is often easier to understand than other programming styles.

If you want to simplify the capitalize function, you can also write it using the fat arrow syntax. If you want to learn a bit ahead, you can watch my Arrow Functions video.

We don't even need to name this function, because an arrow function can also be placed inside the map method as an argument:

```
let words = 'This sentence contains one spelling error.';

words = words
    .split( ' ' )
    .map( w => w.charAt(0).toUpperCase() + w.slice(1) )
    .join( ' ' );

console.log(words);
```

Exercise on String Parsing

Learn to parse strings using the trim and split methods of javascript.

Suppose the following CSV data string is given. Parse the data such that you create an array of lines. Each line is an array of strings containing each data point once.

```
const data = `
Title,Author,Publication Date,Publisher
ES6 in Practice,Zsolt Nagy,2017,Self-published
The Developer's Edge,Zsolt Nagy,2016,Self-published
Regex Quick Syntax Reference,Zsolt Nagy,2018,Apress
The Charismatic Coder,Zsolt Nagy,2018,Self-published
Deep Dive into Functional JavaScript,Zsolt Nagy,2017,PacktPub
Implementing and Testing Applications using Functional JavaScript,Zsolt Nagy,2017,PacktPub
Mastering Functional JavaScript Libraries,Zsolt Nagy,2017,PacktPub
Beginning Modern JavaScript Development with Microservices  WebRTC and React,Zsolt Nagy,2017,
Beginning ASP.NET,Zsolt Nagy,2017,PacktPub
Become the CSS Hero of Your Office with CSS Architecture,Zsolt Nagy,2017,SitePoint
Setting Up and Kick Starting TypeScript,Zsolt Nagy,2017,SitePoint
`;
```

Solution

First, note that there is a newline character before the first line and after the last line. We can get rid of these characters by trimming the string.

We will then split the trimmed string into lines and then split the lines into fields separated by commas. Splitting the string into lines can be done by referencing the newline character \n. Note that the ‘data’ constant has been prepended in a hidden code tab to the following solution to avoid repetition!

```
const result = data.trim().split('\n').map( row => row.split(',') );
console.log(result);
```

Note that there are no commas in the titles. In this specific case, it makes a lot more sense to use another separator character between the fields, as commas may occur in any title. If you encounter such a problem, use a separator that makes the most sense to you.

If you don't want to use a map function, you can also iterate:

```
const result = [];
const rows = data.trim().split( '\n' );
for ( let row of rows ) {
    result.push( row.split( ',' ) );
}

console.log(result);
```

Exercise on String Slicing

Practice your slice function with this exercise.

Sort the rows in the results of Exercise 3 based on their titles. Make sure you exclude the header row.

Hint: the `slice` method works in the same way on arrays as on strings.

Solution

First we get rid of the first row:

```
const result = data.trim().split('\n').map( row => row.split(',') );

const dataRows = result.slice( 1 );
console.log(dataRows)
```

We have to sort the data rows using the array sort method. This method expects a helper function that expects two arguments, `a` and `b`. This function should return a positive value whenever `a` > `b`, a negative value whenever `a` < `b`, and zero if `a` and `b` are equal.

The helper function uses the `localCompare` string method, as it produces a return value in a similar fashion:

```
const result = data.trim().split('\n').map( row => row.split(',') );
const dataRows = result.slice( 1 );
const compareTitles = function( row1, row2 ) {
    return row1[0].localeCompare( row2[0] );
}
dataRows.sort( compareTitles );
console.log(dataRows);
```

Finally, we might want to add the header row on top of the `compareTitles` array. There are at least five different ways to do this, but this time, we will stick to a solution that you already know how to implement:

```
const result = data.trim().split('\n').map( row => row.split(',') );
const dataRows = result.slice( 1 );
const compareTitles = function( row1, row2 ) {
    return row1[0].localeCompare( row2[0] );
}
dataRows.sort( compareTitles );
const dataRowsWithHeader = [ result[0] ];
for ( let row of dataRows ) dataRowsWithHeader.push( row );
console.log(dataRowsWithHeader);
```

Later, once you learn array operations and the spread operator, you can use more concise solutions such as

```
const dataRowsWithHeader = [ result[0], ...dataRows ];
```

or

```
dataRows.unshift( result[0] );
```

The former spreads (enumerates) all elements in `dataRows` and places these elements after `result[0]`. The latter places `result[0]` in front of the elements of `dataRows`. Bear in mind that the last solution mutates (changes) the original `dataRows` array.

Introduction

This lesson explains why it has become so important to learn JavaScript today.

If you have read most of the blog posts of zsoltnagy.eu, you can conclude that most of the articles require at least some basic knowledge about JavaScript. The main exception is the JavaScript Basics category.

I wrote this course to bridge the gap between my less experienced audience and the knowledge required to understand my more advanced content on JavaScript. This course requires absolutely no JavaScript knowledge.

To gain some leverage on learning JavaScript, it is important to see why it is worth learning the language in 2018 and beyond.

- why today is the best time to get started with JavaScript,
- what your career prospects are,
- and why JavaScript provides you with a close to optimal learning experience in 2018.

JavaScript on the Client

This lesson explains how JavaScript is an important technology to use from the client side development perspective.

Client-Side Development:

JavaScript has been clearly the language to go to when it comes to client-side development. Let's stop for a moment. What is client side?

> ***Client-side*** development focuses on writing code that runs in the browser of the end user.

Back in the 90s and early 2000s, JavaScript was a little toy language that could *animate* elements on a static website. These animations were called *dynamic behavior*.

Animation and Rendering:

> ***Animation*** in computer science is the act of moving an object on screen. During the move, the features of the object may change, which means that the animated object may be drawn in different color, different size, or in case of three-dimensional animation, we might see the object from a different angle.

> Displaying an object on screen at a given point in time is called ***rendering***. Animation can also be defined by continuously rendering an object on screen, while its features, including its position, may change.

A Brief History of JavaScript

Twenty years ago, JavaScript was not taken seriously in the software development industry. Many "developers" had no idea what they were doing,

and they just copy-pasted code snippets to add some dynamic behavior to

their site. Often times, these snippets clashed with each other, because they used the same variables.

AJAX Programming:

JavaScript became more popular, once *AJAX* programming (Asynchronous JavaScript and XML) started spreading. In 2006, the `XMLHttpRequest` object became standardized, meaning that you could submit a request to the server and wait for its response. Once the response arrived, you could display the results *without reloading the page*. This worked like magic on websites because before this technique became wide-spread, every interaction with the server required a full page reload. As a page reload may take seconds, getting rid of the need for reloading pages was a big achievement.

The JavaScript community was growing, but we had another problem at hand: in order to write code that works in all major browsers, we had to write different code for different browsers. Sometimes the situation was hopelessly confusing. Therefore, you had to be a **domain** expert to write JavaScript code.

jQuery:

Not for long though. In 2006, another important milestone happened. The library `jQuery` got released with the slogan *write less, do more*. Instead of writing tens of lines of code to make sure you covered all browsers, jQuery gave you single line commands to achieve the same result. You didn't have to be a serious software developer to write JavaScript code. You just had to include jQuery, and your problems were solved.

Unfortunately, `jQuery` created another problems. We could use the slogan of Scarlet Spider, the evil clone of Spider-Man: *all the power, none of the responsibility*. Lowering the entry barriers meant that people who didn't care much about software engineering, also started writing code.

Single Page Applications:

At the same time, the idea of *single page applications* and *client-side rich web-applications* became mainstream. Before AJAX and jQuery, almost everything was on server-side. The task of the server was to assemble a web page, and send it to the client. The server created HTML, CSS, and JavaScript code, and

send it to the client. The server created HTML, CSS, and JavaScript code, and the client only loaded that one page. This changed when the application logic

was moved to the client. This meant the application had to be loaded once, and multiple megabytes of JavaScript code was responsible for rendering the part of the application the client was interested in. This step was made possible by **Moore's law**, because the performance of the end user's computers doubled every few years.

> ***Single-page applications*** receive all the HTML, CSS, and JavaScript on page load, and the client can navigate between different views of the application without informing the server. The client communicates with the server with AJAX calls, and receives data without the need to reload the page. This way, single-page applications became faster, and as easy to use as a desktop application.

Maintainability Issues:

The main problem was that people calling themselves software developers, used their hacking skills to create bad quality software. People soon realized that some jQuery knowledge limits the size of the application they can develop. Beyond a certain size, they encounter maintainability issues.

> ***Maintainability*** is the act of making a software solution easy to develop, modify, scale, test, and debug. If you write ten lines of code, you need a different approach than writing a million lines of code.

JavaScript Frameworks:

The answer to the maintainability issue was the emergence of JavaScript frameworks. Frameworks made it possible to write maintainable code in a structured way, without the need to reinvent the wheel. Frameworks enforced some well known best practices from the field of software-engineering. In order to understand these best practices, the entry barriers got raised again to a healthy level.

> A ***framework*** structures your code and runs your application. When

> using a framework, the framework is *in control*. This means, the framework calls your code.

Library:

> A ***library*** is a set of functionalities that you can call to perform a given task. A library is not a framework, because as long as you use a library, you are in control, the library does not call your code.

In the early 2010s, literally, everyone wanted to write a framework. A new framework got created literally every week. Creating a framework means that you reinvent the wheel, because you solve a well known problem in your own way. If you want to build a portfolio, don't even think about building a framework, because it does not make sense.

Fortunately, we are past the time when the **churn rate** of JavaScript development was so high that you had to learn a new framework or library every month. Today, it is quite clear that you use React or AngularJs or VueJs, and React seems to be winning the battle. There is a big community behind the main frameworks, because React is backed by Facebook, and Angular is backed by Google.

The frameworks were also great, but developers had one more problem. The frameworks and libraries were so much beyond the capabilities of JavaScript that JavaScript became a burden. The language was already quite old, and we needed an update. This update was **ES2015**, or using its old name, **ES6**. Many innovations made it to ES6 from different sources such as `jQuery`, some `frameworks`, some functional programming libraries like `UnderscoreJs`, and some languages like `TypeScript`.

Quick Quiz!

1 Which of the following is not a framework?

Conclusion:

Since ES2015, there was no turning back. Every year, some updates made it to JavaScript, and these updates also ended up in my course, JavaScript in Practice: ES6, ES2017 and beyond.

We can conclude based on the history of JavaScript that there has never been a better time to be a JavaScript developer. In many companies, you can make more money than a C++ developer or a PHP developer. The challenges you deal with on a daily basis are interesting, and more importantly, you often get immediate feedback on your efforts by seeing the effects of your changes in your browser.

Summary:

- JavaScript was first a toy language for creating dynamic behavior
- AJAX made it possible to update websites without reloading the page
- jQuery lowered the entry barriers to programming
- Frameworks and libraries forced us to use well-known design patterns in software engineering
- ES2015 and beyond: many patterns became a standard part of the language

JavaScript on the Server

This lesson explains how JavaScript is an important technology to use from the server side development perspective.

If these benefits were not enough for you, let's add some more sauce to the mix. Let's first focus on the server-side programming.

> ***Server-side*** development focuses on writing code that runs on the server of the service provider.

`Node.js` and Server-Side Development

> JavaScript is not just a client-side language anymore.

`Node.js` makes it possible to run JavaScript on the server. `Node.js` is built on the foundations of the execution engine of Google Chrome, called V8. The V8 engine is fast like a Formula One car. The V8 compiler turns JavaScript to machine code, which means that on the server, we are not dealing with *interpreted* code anymore.

> Interpreted code runs on a *virtual machine*, translating the sets of instructions written by a developer to executable commands. Interpreted languages are typically safer, but slower than *compiled* languages. Interpreted languages are also *portable*, which means that you can take the source code of a program and execute it on any system that has a virtual machine for that specific code. Languages that compile to *machine code* have the benefit of running code as close to machine code level as possible. This translates to faster execution times. Unfortunately, this machine code cannot be *ported* to different machines running different operating systems or having different hardware architecture.

> architecture.

Node.js solved the speed problem by offering a compiler to machine code. This is not a big deal yet, because most other server-side languages can do the same thing.

Why is `node.js` still a good choice for certain server-side tasks?

The reason lies in some `node.js` architectural decisions. `Node.js` comes with an **event loop** that can execute and serve parallel requests on the server without blocking. This is called *non-blocking I/O*. Opposed to many other languages, when you have to read or write a database or a file, node.js is not blocked until the I/O operation is finished. This means that `node.js` stays busy handling other tasks, possibly important for other clients.

Think about it for a moment. When is this feature beneficial?

> `Node.js` shines when there are a lot of independent parallel requests requiring a lot of I/O operations, but not many server-side computations.

For instance, 3D renderer algorithm comes with a lot of CPU operations. Even though the V8 engine produces fast machine code, there are some other languages that are faster. As those languages have a ***blocking*** nature when it comes to I/O operations, `node.js` gets the edge over them once we deal with many inexpensive operations that interact with a database.

In today's world, this use case happens to be very popular. The computers of the end users are strong enough to run a lot of logic on client-side. The application server providing an API mostly interfaces database servers and asset servers, and formats data in a way that clients can fetch and understand them using AJAX requests. The role of the server is limited to purposes that do not require a lot of CPU operations. This is an optimal environment of node.js.

This is because a lot of problems have been solved effectively by **microservices**. You can simply query a microservice to perform a CPU-heavy operation, often on another server, using a different technology. If you are

interested in microservices, check out my introduction on this topic.

Quick Quiz!

1 In node.js, everything is asynchronous.

COMPLETED 0%

1 of 2

Conclusion:

As a conclusion, once you learn client-side development, you can upgrade your skills to perform server-side development as well. Developers who are good at both client side and server side development are called *full stack developers*. Full stack development is a great career path upgrade for you in case you are looking for more responsibility in a smaller company.

Summary:

- JavaScript is compiled to fast machine code on the server, using the V8 engine of Google
- Node.js comes with non-blocking I/O, making it possible to handle many I/O operations in parallel

Other Exotic JavaScript Use Cases

Let's look at some other benefits of using JavaScript.

JavaScript development is fun!

The primary use case of JavaScript has been client side development for web-browsers. Then came node.js with the ability to write server-side code. JavaScript has a lot more to give in 2018 though. Let's see some of the more exotic options:

- **JavaScript and VR.** React 360 makes it possible to deliver VR experiences using JavaScript. Some other libraries help you with VR as well.
- **JavaScript can compile to other languages, and other languages can be compiled to JavaScript.** *WebAssembly* makes it possible to compile many languages to JavaScript so that you can run code in the browser.
- **Machine Learning in JavaScript.** TensorFlow, Google's machine learning library containing neural network implementations, is also available in JavaScript. Many other libraries for machine learning and artificial intelligence are also available.
- **Mobile development.** React Native provides a way to perform mobile development using JavaScript. Native mobile apps and JavaScript are an amazing combination, because of the facilitation of code reusability.
- **Blockchain Programming.** At the time of writing this article, there are more than a thousand blockchain-related libraries on npm (proof). Crypto-js provides us with easy-to-use encryption and decryption algorithms. Everything is set to implement blockchains in JavaScript.
- **Real Time Communication on the Web**: video and audio streams can be controlled using JavaScript. The WebRTC open source project makes it possible to create applications with real time communication.
- **Microservices**. Seneca is one example for organizing microservices in your application.

If you are an intermediate to advanced JavaScript developer, and you are interested in the latter two use cases, check out my video course on Packt Publishing: Beginning Modern JavaScript Development with Microservices, WebRTC, and React.

JavaScript may or may not be optimal for certain tasks. Obviously, if a task is computation intensive, such as character animation and shading in three dimensions, JavaScript may not be the optimal language to write applications that perform these operations. However, JavaScript execution environments are continuously improving, and new use cases emerge on the horizon, where using JavaScript is an alternative. Remember, if a use case becomes feasible in JavaScript, chances are, a big passionate community will soon be formed around it.

JavaScript Domination on the Job Market

- JavaScript and JavaScript frameworks are popular in the industry
- Demand for JavaScript developers is higher than the supply

According to the 2018 survey of HackerRank:

- JavaScript is the number 1 language employers are looking for. In total, 47.8% of employers post JavaScript positions
- There is a big gap between employer demand and employee knowledge in JavaScript frameworks. The biggest gap is with React: 33.2% of employers are looking for React developers, while the supply is only 19%. Scarcity translates to more money, less competition, and more opportunities.
- You can be a self-taught developer, because companies place significantly more emphasis on experience (90.6%), your portfolio (72.7%) than on education. If you market it properly, experience can be gathered developing your personal projects, which will end up in your portfolio. This means, in JavaScript development, only your efforts limit you, there are no gatekeepers, universities, internships, or certification industries. In Berlin/Germany for instance, you can become a self-taught programmer on your own, and you can get hired for a higher salary than the average salary for a German citizen. In case of many cities or remote work opportunities, the same principle applies.
- Although JavaScript only ranks fifth in languages developers prefer behind Python, C, C++, and Java, this might be offset by the bad experience coming from older people who had to deal with the immature version of JavaScript. When it comes to developer experience, an important metric is that out of all frameworks of all languages, the top four are JavaScript frameworks: Node.js, React, ExpressJs, and AngularJs. Working with these frameworks makes developers happier than anything else.

The 2018 survey of StackOverflow paints the same picture:

- JavaScript is the most popular language with a share of 69.8%. On StackOverflow, JavaScript is more popular than Python.
- Three frameworks lead the list of most popular frameworks: Node.js, Angular, React.
- Salaries of JavaScript developers are marked lower in this survey than the salary paid in many other fields. In my opinion, this statistic is misleading, because in the case of a language requiring a lot of expertise, the salaries of experts are overshadowed by the salaries of entry level developers. In the average first JavaScript interview, based on my experience, it is not uncommon to filter out 70% of the applicants. In an average C++ interview on the other hand, people who reach the first interview know a lot better what they are doing. I encourage you to do your own research on payscale.com or glassdoor.com to find specific JavaScript developer salaries of companies that require your skills.

Although the audience matters, in today's world, you cannot really go wrong with JavaScript. You can also expect that JavaScript stays in an uptrend.

In the next lesson, we'll examine why the rapidly growing JavaScript language is very easy to learn.

Why Is It Easier Than Ever To Become A JavaScript Developer?

JavaScript has greatly evolved since its first launch. Various aspects like 'browser dev tools' and 'third party library support' make it much easier to get familiar with JavaScript.

We have already dealt with the economical aspect of becoming a JavaScript developer. There is no doubt about the high demand for JavaScript developers on both client and server sides.

There is no money in the world that would compensate you for a bad developer experience though.

Learning JavaScript is easier than ever before. Years ago, a lot of frustrations prevented an average developer from experiencing an optimal learning experience:

- Computer resources were more scarce than now
- The standardization of the language was a lot more immature than now, yielding to if-else structures containing different code snippets for each browser
- The ES5 standard provided a lot worse developer experience than ES2015 and beyond
- JavaScript was not regarded as a serious programming language
- Browser developer tools and debugging capabilities were immature
- Third party library support was scarce and unreliable
- Good integrated development environments or text editors did not exist

Talk to anyone who wrote JavaScript code fifteen years ago, and you can conclude, you are not working with the same language anymore.

If there is a problem, a passionate open source community, as well as tech giants like Google or Facebook, are likely to provide you with a reliable JavaScript solution. Since 2015, standardization also started speeding up. We are living in the greatest time of JavaScript history. And it's just going to be better.

It is obvious that Javascript will become easier as time as goes by, but what kind of learning curve should you expect in this language? Well, there are scientific reasons for why the JS learning experience is better than many other languages.

Go to the next lesson to find out why this is the case.

JavaScript Learning Experience According to Science

Let's take a look at the scientific criteria for a learning experience to be classified as good. We'll also examine where JavaScript falls in terms of these criteria.

The first good news is that you can immediately start writing code. The second good news is that you can see the effect of your efforts within a seconds. You don't need to go through a long compilation process. The things you build with JavaScript often manifest in visible changes on screen.

While I am not claiming that learning JavaScript is super easy, I do claim that you get more direct and immediate feedback when dealing with JavaScript than with most other languages. Why does this matter?

Because everything is given for you to experience flow. Flow experiences are often referred to as optimal experiences. It feels like you are *in the zone*. If you are in flow, time flies. You are deeply immersed in an activity. Your learning experience becomes close to optimal.

According to science, there are a few necessary conditions for flow to happen:

1. The goals of the activity must be clear
2. The task is neither too easy, nor too hard
3. You get direct feedback on the outcome of your actions

Let's look at these points one by one.

The Goals Of The Activity Must Be Clear

Regarding the goals, just look at the two developer surveys and figure out whether it makes sense to become a software developer in 2018. You can literally work from the beach as a freelancer, or as an employee for an international team. Writing JavaScript code often feels like playing with LEGO. You also happen to make good money with it. Within half a year, many people become productive as junior developers. If you can't write code right

now, what better experience could you imagine for yourself than utilising your skills as a frontend or full stack developer?

The Task Is Neither Too Easy, Nor Too Hard

Learning JavaScript is not an obvious thing to do. There are some riddles, pitfalls and hurdles you need to go through. Don't let this discourage you though! Just think about it. Who would you be if you never faced a challenge? You would have nothing in your life to be proud of. These hurdles serve to benefit you. These hurdles also make your learning experience appropriately hard, so that you can experience flow while learning JavaScript.

My opinion is that when learning JavaScript, it is not too hard to experience flow. Obviously, this opinion is debatable, because if you worked as a bus driver for instance, and you have never seen a programming language before, you will need some fundamentals. I admit that this course is not an optimal resource for absolute beginners, because it assumes you know at least the basics of how programming languages work e.g. what **if statements** and **loops** are, what functions are etc. I am planning an entry level product for JavaScript, and I know, advanced JavaScript may be too hard for some people.

However, once you lay down the foundations, you will be ready for these resources, and nothing will stop you. Therefore, with proper guidance, you will improve without getting frustrated about your learning experience.

You get direct feedback on the outcome of your actions

This is where JavaScript shines. As long as you use JavaScript on the client side without complicating your tooling too much, you will get immediate feedback on your actions. Sometimes you just have to refresh your browser. In some cases, you don't even need a browser refresh. Whatever you do, you will immediately see the results of your code.

For this reason, JavaScript is a great candidate for flow experience.

Summary

An overview of how and why the future for JavaScript developers looks bright.

Regardless of whether you simply want to learn JavaScript, or change your specialization, it is important to take a moment and appreciate the current state of JavaScript.

Things were not always as bright they are today. In the first half of this section, we went through the dark history of JavaScript on client side, then we transitioned to the somewhat brighter and shorter history of JavaScript on the server.

These events made it possible for JavaScript developers to experience growth way beyond what we imagined ten years ago. An avalanche of use cases await JavaScript developers, such as VR and Blockchain technologies, helping JavaScript conquer new horizons.

As JavaScript continuously gains popularity, software developers who specialize in technologies that include JavaScript have a bright present and future. Two developer surveys concluded that you cannot really go wrong while betting on JavaScript.

If you would like to learn JavaScript, the last part of the last section concluded that JavaScript is close to optimal in terms of providing you an optimal learning experience. Given that you often experience an immediate feedback to your actions (as long as you stick to reasonable challenges), you can expect to spend more development and learning time in JavaScript compared to many other languages.

Creating the Markup

This lesson will introduce you to the HTML markup language and teach you how to create a simple example form.

Introduction

Previously, you have learned how to write basic JavaScript code. In this chapter, you will find out how to use JavaScript with HTML and CSS.

The goal of this chapter is not to teach you the basics of CSS or JavaScript button click event handling, but to show you how HTML, CSS, and JavaScript is structured.

Creating the Markup

When visiting a website, we define its HTML markup, some styling information, and some dynamic functionality.

Markup is written in HTML. HTML stands for Hyper-Text Markup Language. We will not introduce HTML in detail, so in case you don't know how to write simple HTML documents, check out this article followed by this tutorial.

Let's create a simple example with a form. We will place a textfield in a form, and a button that displays a greeting in the form of an alert box.

Output

JavaScript

HTML

CSS (SCSS)

```
<!doctype html>
<html lang="en">
  <head>
    <title>Greetings</title>
  </head>
  </body>
    <form>
      <input type="text"
```

```
      <input type="text"
             class="tf-large  js-name"
             placeholder="name">
      <button class="btn-large  js-hello">Greet</button>
    </form>
  </body>
</html>
```

If you click on the Output tab, it renders your HTML in that window. You can see a textfield there, where you can enter your name. Unfortunately, when you press the Greet button, the text you enter in the textfield is gone. This is what we will fix with JavaScript next.

Adding JavaScript Code

This lesson will teach you how to add Javascript to build functionality of the HTML form.

When you create an HTML form, pressing a button submits the form by default. Submitting a form reloads the page. Once you reload the page, the data entered in the textfield is lost. This is what we will prevent now.

Output

JavaScript

HTML

CSS (SCSS)

```
function helloListener( event ) {
    event.preventDefault();
    console.log( 'button pressed' );
}

const helloButton = document.querySelector( '.js-hello' );
helloButton.addEventListener( 'click', helloListener );
```

Console

Clear

First, we created a `helloListener` function. This function prevents the default action of the event, which is the submission of the form.

The second line in the function creates a console log that appears in the developer tools of your browser. More on this later.

The last line attaches the `helloListener` function to the button. We tend to use `js-` prefixed classes to refer to elements in the Document Object Model, also

known as the DOM. The `document.querySelector` function takes a selector

string, in this case a class name, and locates the node in the DOM that has this class. Check out the HTML tab, you can see the same class in the class list of the button.

Once we located the `.js-hello` button, we can add an event listener function to it. This function takes an event, and can perform any JavaScript action ranging from manipulating the contents you can see on screen to calling a service or an API on the web, saving your data in a persistent storage.

If you will work on your computer, you will face one problem:
You will not have access to Javascript code in the HTML file. You will have to add an HTML tag

```
<script src="yourjsfilename.js"> </script>
```

at the bottom of the body.

Developer Tools

Each browser has developer tools. For example, in Google Chrome right clicking on your website anywhere inside the browser window, and selecting `Inspect` from the context menu, you will find yourself inside the developer tools. Find the `Console` tab.

Assuming you have clicked on the button, you can find the following there:

```
button pressed
```

Your console logs will also be displayed underneath your code.

In the Developer Tools, You can execute any JavaScript expression by writing it after the `>` prompt:

```
 button pressed
> 2+2
4
> helloButton
 <button class="btn-large js-hello">Greet</button>
```

The `helloButton` variable stores a DOM node, fully accessible in JavaScript.

Finishing up with Style

In this lesson we finish up the functionality and give you a brief overview of CSS Styling.

Greeting the User

If you enter `document.querySelector( '.js-name' ).value` in the console, you can log the value entered in the text-box.

Instead of a console log, we need to get the value of the textfield and display a greeting in an alert box:

Output

JavaScript

HTML

CSS (SCSS)

```
function helloListener( event ) {
    event.preventDefault();
    const name = document.querySelector( '.js-name' ).value;
    alert( 'Hello, ' + name + '!' );
}

const helloButton = document.querySelector( '.js-hello' );
helloButton.addEventListener( 'click', helloListener );
```

The `+` operator on strings concatenates strings. The `alert` function creates an alert box.

If you test the code by entering a name, you can see that everything is in place.

Styling

We know that

- The static markup providing information on the structure of the website is in the HTML file,
- The dynamic functionality goes in the JavaScript files.

Many people think that the look and feel of the page is also defined in the HTML file. This approach is wrong. Styling is separated from HTML.

We describe styles in CSS (Cascading Style Sheets) files.

You can see the styles defined, and the updated output below.

Output

JavaScript

HTML

CSS (SCSS)

```
.tf-large {
    font-size: 1.5rem;
    padding: 1rem;
}

.btn-large {
    font-size: 1.5rem;
    padding: 1rem;
    font-weight: bold;
}
```

Similarly to JavaScript, we also need to reference the CSS file from the HTML document. We do it in the head using a `<link>` tag:

```
<link rel="stylesheet" href="styles.css">
```

HTML Classes: Separation of Concerns

In this lesson we teach you some design patterns to use when handling numerous HTML elements and also multiple JS and CSS files.

Recall the `button <button class="btn-large js-hello">Greet</button>`. You can see two classes in there, `btn-large` and `js-hello`. The first class is used solely for styling, and the second class is used solely for referencing in JavaScript.

No-one forces you to write code this way, but it pays off to separate classes for styling and functionality. When multiple people work on the same codebase, this separation pays off. This way, a person responsible for styling can add, delete, or rename any styling classes without affecting functionality. JavaScript developers can do the same thing with the `js-` prefixed classes.

> `js-` prefixed classes should be used for functionality, while regular classes should be used for styling.

Why aren't we using HTML IDs?

You may know from your studies or previous work that we can also reference DOM nodes using their ID attributes:

```
<div id="intro">This is an introduction</div>
```

If you have the above node in the DOM, you can reference it using

```
document.getElementById('intro')
```

The problem with ID attributes is that they have to be unique for the whole document. If you violate this rule, you get an HTML error.

In big websites and applications, most developers never know the context in which their markup will appear. Chances are that if you use an ID attribute

name, someone else will do the same elsewhere.

> As two DOM nodes cannot have the same ID, using ID attributes is not advised in most cases.

Once a business owner asked me to try out his affiliate plugin, because it was not working for him. Sure enough, I filled in the form, but once I pressed register, nothing happened. I checked the developer tools of the browser, and it turned out that there were some duplicated ID attributes in the markup.

I asked him how many times he included the affiliate plugin. He said, once for desktop computers, and once for mobile. He proudly told me that he made the mobile version hidden when someone checks it in a desktop.

Unfortunately, he only managed to hide it using a CSS style (`display: none`). The markup was in his document with the exact same ID attributes as the other version.

```
<div id="affiliate-plugin" class="desktop">
    <!-- Content goes here -->
</div>
<div id="affiliate-plugin" class="mobile  hidden">
    <!-- Content goes here -->
</div>
```

The above markup is erroneous due to the duplicated ID attributes. My entrepreneur friend, lacking web development knowledge, lost hours on not knowing the fundamentals. You can now save these hours next time you encounter a similar situation.

What do we do with multiple JavaScript and CSS files?

In large projects, placing all the JavaScript code in one file is not advised for development. Therefore, we often have hundreds if not thousands of JavaScript and CSS files.

Placing all these files in the HTML markup is not advised either, because enumerating hundreds of files in the markup is neither convenient, nor efficient. You may also encounter JavaScript errors if you include the JavaScript files in the wrong order.

> For large projects, use Webpack and npm (Node Package Manager) to bundle your files into one file that you can include in your markup.

In a large project, I highly recommend that you learn and use SASS. SASS not only enriches your CSS syntax, but it helps you structure it better.